



UNIVERSITÀ DEGLI STUDI DI SALERNO

Dipartimento di Informatica

Corso di Laurea Triennale in Informatica

TESI DI LAUREA

Titolo della Tesi

RELATORE

Prof. Fabio Palomba

Dott. Nome tutor

Università degli Studi di Salerno

CANDIDATO

Marco Donatiello

Matricola: 0512119045

Anno Accademico 2025-2026

Questa tesi è stata realizzata nel

sesa^{lab}
SOFTWARE ENGINEERING
SALERNO

Dedica o citazione

Abstract

Il testo dell'abstract qui...

Indice

Elenco delle Figure	iii
Elenco delle Tabelle	iv
1 Introduzione	1
2 Analisi dello stato del arte e contesto tecnologico	2
2.1 OWASP Cloud Security Top 10	2
2.1.1 Security Misconfiguration	4
2.2 API Security Risks	6
2.2.1 API1:2023 – Broken Object Level Authorization	8
2.2.2 API2:2023 – Broken Authentication	15
2.2.3 API3:2023 – Broken Object Property Level Authorization	17
2.2.4 API4:2023 – Unrestricted Resource Consumption	20
2.2.5 API5:2023 – Broken Function Level Authorization	22
2.2.6 API6:2023 – Unrestricted Access to Sensitive Business Flows	25
2.2.7 API7:2023 – Server Side Request Forgery	27
2.2.8 API8:2023 – Security Misconfiguration	29
2.2.9 API9:2023 – Improper Inventory Management	32
2.2.10 API10:2023 – Unsafe Consumption of APIs	34

3	Metodo di ricerca	37
3.1	Approccio metodologico	37
3.1.1	Obiettivi e domande di ricerca	37
3.1.2	Impostazione dello studio	38
3.1.3	Il criterio di classificazione	39
3.1.4	Requisiti derivati per lo strumento di rilevazione	40
3.1.5	Disegno della valutazione	41
3.2	Rilevamento delle Security Misconfiguration	42
3.2.1	Architettura del sistema	42
3.2.2	Analisi del codice applicativo	44
3.2.3	Analisi delle definizioni infrastrutturali	47
3.2.4	Analisi della specifica dell'interfaccia	47
3.2.5	Correlazione e punteggio di rischio	48
3.2.6	Validazione e limiti	50
4	Conclusioni	53
	Bibliografia	54

Elenco delle figure

Elenco delle tabelle

3.1	Regole di analisi del codice applicativo per la Security Misconfiguration. La confidenza è quella assegnata prima del filtro a 0,70.	45
-----	--	----

CAPITOLO 1

Introduzione

Alcune linee guida per scrivere la tesi: 1) Ogni immagine/tabella deve avere le seguenti caratteristiche: a) Una caption; b) Deve essere esplicitamente commentata all'interno del testo (ricordatevi che se vi riferite alla figura/tabella inserendo anche il corrispettivo numero e.g., Figura X mostra che... allora la prima lettera deve essere scritta in maiuscolo; E ricordatevi che latex mette a disposizione un comando per citare correttamente (ref) senza dover mettere a mano il numeretto corrispondente) c) Le immagini devono essere rifatte tramite PPT o similari e salvate in pdf prima di inserirle nel file latex, altrimenti rischiate di inserire immagini sgranate. 2) Ricordatevi sempre di citare le fonti e di rifrasare, altrimenti rischiate di incorrere nel reato di plagio (che può comportare anche la revoca del titolo accademico) e che citare non sminuisce il vostro lavoro, anzi, lo rafforza. 3) L'Abstract deve necessariamente contenere le seguenti informazioni a) Un breve riassunto della problematica; b) Una breve descrizione di ciò che è stato fatto; c) I risultati ottenuti.

Analisi dello stato del arte e contesto tecnologico

2.1 OWASP Cloud Security Top 10

Il portale AppSecMaster ha elaborato una propria classificazione dei dieci rischi più comuni e di maggiore impatto nei moderni ambienti cloud, ispirandosi ai principi promossi da OWASP in materia di sicurezza applicativa, che vanno dall'infrastruttura mal configurata alla scarsa visibilità delle risorse in esecuzione. Di seguito è riportata la lista delle principali problematiche individuate:

1. **Security Misconfiguration** — Bucket di storage lasciati con permessi troppo permissivi o istanze di calcolo distribuite senza le dovute restrizioni restano tra le cause più frequenti di violazione dei dati in ambienti cloud.
2. **Identity / Access Mismanagement** — Ruoli IAM assegnati con privilegi superiori a quelli effettivamente necessari, e mai rivisti dopo la configurazione iniziale, espongono i sistemi a un concreto rischio di escalation dei permessi.
3. **Improper Secrets/Key Management** - Se chiavi API e credenziali vengono salvate in chiaro o incorporate nel codice sorgente, la loro compromissione può aprire un accesso laterale a più servizi collegati.

4. **Insecure Network Segmentation** — Senza una reale separazione tra reti e ambienti diversi, un attaccante che viola un singolo componente può muoversi liberamente verso risorse più sensibili.
5. **Injection / Input Validation Flaws in Cloud Services** — Le classiche vulnerabilità di injection assumono forme nuove nei contesti cloud-native, colpendo funzioni serverless, code di messaggi e gateway API dove i controlli sugli input non sono sempre applicati in modo uniforme.
6. **Insecure Deserialization / API Exposure** — La gestione non sicura di dati serializzati, unita ad API interne prive di adeguati controlli di autenticazione o versionamento, costituisce un punto d'ingresso sfruttabile per eseguire codice non autorizzato.
7. **Vulnerable Third-Party/Container Dependencies** — Librerie open source e immagini container non aggiornate accumulano vulnerabilità note, soprattutto quando le stesse basi vengono riutilizzate tra più progetti senza verifiche periodiche.
8. **Improper Asset Inventory / Visibility** — Risorse cloud dimenticate o mai tracciate correttamente, incluse istanze abbandonate e servizi "ombra", sfuggono facilmente al controllo dei team di sicurezza.
9. **Insufficient Logging / Monitoring** — Log distribuiti su decine di servizi diversi e privi di un formato coerente rendono difficile individuare per tempo un'intrusione in corso.
10. **Unvalidated Redirects/Cloud Function Abuse** — Funzioni serverless eseguite con permessi troppo ampi, combinate a controlli insufficienti sui redirect, possono essere sfruttate per aggirare le restrizioni di accesso e ottenere un punto d'appoggio nascosto nell'infrastruttura.

Tra le problematiche elencate, il presente lavoro si concentra sulla prima, Security Misconfiguration, e sulla componente di esposizione delle API della sesta, Insecure Deserialization / API Exposure: sono le due voci più direttamente legate agli obiettivi del sistema sviluppato e quelle che ricorrono con maggiore frequenza negli scenari di

autorizzazione tipici delle architetture basate su API. La prima è approfondita nella sottosezione seguente; la seconda nella Sezione 2.2, dove la classificazione OWASP dedicata alle API la articola in dieci categorie distinte.

2.1.1 Security Misconfiguration

Una misconfiguration, in ambito cloud, nasce da un errore nella configurazione di un servizio, di un prodotto o di una risorsa infrastrutturale: le impostazioni di sicurezza predefinite non vengono irrobustite a sufficienza, e questo lascia scoperto un fianco che con poco sforzo potrebbe essere protetto. I numeri parlano chiaro: il documento ufficiale OWASP riporta che il 90% delle applicazioni testate è risultato vulnerabile ad almeno una forma di misconfiguration [1]. Nella maggior parte dei casi non c'è nulla di sofisticato dietro: è una dimenticanza, un'impostazione lasciata di default, un permesso che nessuno ha mai rivisto. Ed è proprio questo che la distingue da una vulnerabilità nel codice applicativo — qui il problema non sta nella logica del software, ma in come le risorse sono state predisposte.

Le forme più comuni in cui una misconfiguration si manifesta sono, in fondo, poche e ricorrenti. Un bucket di storage lasciato accessibile pubblicamente, quando avrebbe dovuto restare privato, è probabilmente la più diffusa: basta una policy impostata male, magari copiata da un altro progetto senza troppa attenzione. Un container che gira con privilegi di root, invece, espone il sistema a un rischio diverso ma altrettanto concreto — se l'attaccante riesce a comprometterlo, si ritrova con permessi ben oltre quelli che gli servirebbero per fare danni. C'è poi il caso dei template Infrastructure-as-Code scritti senza le dovute verifiche: un file Terraform o CloudFormation che definisce una risorsa con permessi troppo ampi si porta dietro lo stesso errore ogni volta che viene riutilizzato, replicandolo su ambienti diversi senza che nessuno se ne accorga.

Un caso reale: il data breach Capital One Il caso Capital One del luglio 2019 resta probabilmente l'esempio più citato di questa categoria di rischio. Un bucket Amazon S3 lasciato con permessi troppo ampi permise a un'attaccante di sottrarre oltre 100 milioni di record clienti, tra dati anagrafici, indirizzi e informazioni finanziarie

sensibili [2]. Nessuna falla nella piattaforma cloud era coinvolta: la responsabilità ricadeva interamente sulla configurazione impostata dal cliente. Non a caso AWS, commentando l'accaduto, ribadì che *"Amazon S3 is secure by default"* [3], scaricando di fatto la colpa sulla gestione dei permessi lato utente. Sul piano economico il conto fu salato: 80 milioni di dollari di multa dall'Office of the Comptroller of the Currency, più 190 milioni versati per chiudere una class action. Capital One dimostra bene quanto una semplice disattenzione in fase di configurazione possa costare quanto, se non più di, un attacco costruito su un exploit tecnicamente elaborato.

Episodi come questo spiegano perché, negli ultimi anni, le organizzazioni abbiano iniziato ad affidarsi sempre meno alla revisione manuale delle configurazioni e sempre più a strumenti automatizzati capaci di individuare questo tipo di errori prima che vengano sfruttati. Una configurazione insicura, del resto, raramente viene introdotta di proposito: emerge quasi sempre da un template riutilizzato senza controllo, da un default mai cambiato, da una revisione saltata per mancanza di tempo — fattori che un controllo manuale sistematico difficilmente riesce a intercettare su larga scala e con continuità.

Il documento ufficiale OWASP illustra questa categoria di rischio attraverso quattro scenari esemplificativi [1]. Il primo riguarda applicazioni di esempio dimenticate su un server di produzione, spesso accompagnate da account amministrativi con credenziali mai modificate rispetto ai valori predefiniti. Il secondo descrive un server su cui la funzionalità di directory listing resta abilitata, consentendo a un attaccante di risalire fino al codice sorgente tramite reverse engineering. Il terzo riguarda messaggi di errore troppo dettagliati restituiti dall'application server, capaci di esporre informazioni sull'infrastruttura sottostante o versioni di componenti vulnerabili. Il quarto, infine, riguarda permessi di condivisione predefiniti di un cloud service provider lasciati aperti ad altri utenti su Internet, con conseguente esposizione di dati sensibili archiviati nello storage cloud.

Fra i quattro scenari, due ricadono nel perimetro del sistema presentato in questo lavoro e due ne restano fuori. Rientrano il secondo e il quarto, perché in entrambi la configurazione difettosa è dichiarata in un artefatto ispezionabile — la direttiva che abilita il directory listing nella configurazione del server, i permessi di condivisione

fissati nel manifest della risorsa di storage — e può quindi essere individuata prima del rilascio. Restano fuori il primo e il terzo: le applicazioni di esempio dimenticate in produzione non compaiono in alcun artefatto del progetto e presuppongono un inventario degli host effettivamente in esecuzione, mentre i messaggi di errore troppo dettagliati emergono soltanto dalle risposte prodotte a runtime.

2.2 API Security Risks

Le API oggi fanno gran parte del lavoro che una volta faceva il sito web stesso: non restituiscono una pagina pensata per essere letta da una persona, ma dati grezzi destinati a un'altra app, a un altro servizio, a un sistema che poi decide cosa farne. Lo schema delle richieste è prevedibile, gli endpoint¹ espongono operazioni precise su risorse precise. Per un attaccante questo significa meno lavoro di interpretazione: non deve capire un'interfaccia grafica, gli basta osservare come sono costruite le chiamate.

OWASP mantiene, accanto al Top 10 per le applicazioni web, una classificazione separata dedicata alle API. Alcune categorie compaiono in entrambe le liste — la misconfiguration, il broken access control — ma buona parte dei rischi specifici delle API riguarda un livello che il Top 10 generale non copre: l'autorizzazione su singoli oggetti, proprietà, funzioni. Un'applicazione web tradizionale filtra cosa mostrare all'utente; un'API deve filtrare cosa restituire nella risposta. La differenza sembra minima, ma è da lì che nascono molte delle vulnerabilità trattate in questa sezione.

Le dieci categorie che compongono l'edizione 2023 riflettono questa logica: non rischi generici, ma rischi legati al modo specifico in cui un'API espone dati e funzionalità. Di seguito la lista completa, in ordine di gravità secondo OWASP [4]:

1. **API1:2023 – Broken Object Level Authorization (BOLA)** — un endpoint accetta l'ID di una risorsa senza verificare che chi lo richiede abbia davvero il diritto di accedervi; basta cambiare un numero nell'URL per leggere dati altrui.

¹*Endpoint*: il singolo punto di accesso di un'API, identificato da un URL e da un metodo HTTP, che espone una specifica operazione su una risorsa.

2. **API2:2023 – Broken Authentication** — meccanismi di autenticazione implementati male, token che non scadono mai, o password recuperabili senza controlli sufficienti.
3. **API3:2023 – Broken Object Property Level Authorization** — qui il problema non è accedere alla risorsa sbagliata, ma vedere o modificare campi che non dovrebbero essere esposti o scrivibili da quel particolare utente.
4. **API4:2023 – Unrestricted Resource Consumption** — assenza di limiti su chiamate, dimensione dei payload o uso di risorse computazionali, con conseguenze che vanno dal semplice rallentamento a un vero e proprio denial of service.
5. **API5:2023 – Broken Function Level Authorization** — un utente normale riesce a invocare funzioni riservate agli amministratori, semplicemente perché il controllo sui permessi manca a livello di endpoint o di metodo HTTP.
6. **API6:2023 – Unrestricted Access to Sensitive Business Flows** — la singola richiesta può essere anche legittima, ma se ripetuta migliaia di volte automatizza un abuso della logica di business (es. acquisto massivo di biglietti, creazione automatica di account).
7. **API7:2023 – Server Side Request Forgery (SSRF)** — l'API accetta un URL fornito dall'utente e lo interroga senza validarlo, permettendo di raggiungere risorse interne altrimenti non accessibili dall'esterno.
8. **API8:2023 – Security Misconfiguration** — header di sicurezza mancanti, CORS² troppo permissivo, ambienti di debug ancora attivi in produzione: stessa famiglia di errori vista nel capitolo precedente, qui applicata allo strato applicativo.
9. **API9:2023 – Improper Inventory Management** — endpoint vecchi, versioni deprecate o ambienti di staging dimenticati online, mai rimossi né monitorati, restano comunque raggiungibili da chi li trova.

²CORS (*Cross-Origin Resource Sharing*): il meccanismo con cui un server dichiara quali siti di origine diversa possono leggere le sue risposte dal browser. Una politica permissiva le rende leggibili a qualunque origine.

10. **API10:2023 – Unsafe Consumption of APIs** — la vulnerabilità non nasce nell'API stessa, ma nella fiducia cieca riposta nei dati restituiti da API di terze parti integrate nel sistema.

Tra le dieci categorie appena elencate la prima merita un'attenzione diversa dalle altre: non tanto per la posizione in classifica, quanto per ciò che rappresenta, cioè un controllo di accesso che manca proprio dove ci si aspetterebbe fosse più solido, sulla singola risorsa richiesta. È il tipo di errore che si insinua facilmente in un endpoint scritto in fretta e che, allo stesso tempo, può esporre porzioni estese della base dati di un sistema con una richiesta HTTP modificata di poco. La sottosezione seguente lo esamina nel dettaglio: come si manifesta, da quali cause nasce e con quali misure lo si previene.

2.2.1 API1:2023 – Broken Object Level Authorization

Definizione e collocazione La Broken Object Level Authorization (BOLA) si manifesta quando un'applicazione, pur autenticando correttamente l'utente che effettua una richiesta, non verifica se quello stesso utente abbia effettivamente il diritto di accedere all'oggetto specifico richiesto [5].

Un'interfaccia di programmazione applicativa espone tipicamente un insieme di risorse — utenti, ordini, documenti, transazioni — identificabili singolarmente attraverso un riferimento univoco. L'accesso a tali risorse presuppone due verifiche concettualmente distinte, spesso confuse nella pratica implementativa: l'autenticazione, che accerta l'identità del soggetto che effettua la richiesta, e l'autorizzazione, che stabilisce se quel soggetto, una volta identificato, abbia il diritto di operare su una specifica risorsa.

Un utente autenticato correttamente non è, per definizione, un utente autorizzato ad accedere a qualunque oggetto gestito dal sistema. Questa distinzione, apparentemente elementare, è proprio il punto in cui molte implementazioni falliscono: il possesso di credenziali valide viene talvolta considerato — implicitamente, nel disegno del codice — condizione sufficiente per l'accesso a una risorsa, quando in realtà rappresenta soltanto la premessa per una verifica ulteriore, relativa alla relazione specifica tra quell'utente e quell'oggetto.

Funzionamento delle API e ruolo degli identificatori di risorsa Le API REST, e in misura diversa quelle basate su GraphQL³, espongono le risorse attraverso identificatori che compaiono nel percorso della richiesta, nei parametri di query, nel corpo del messaggio o negli header. Un endpoint come `GET /api/orders/482` restituisce, nella maggior parte delle implementazioni, l'oggetto ordine identificato dal valore 482, indipendentemente dal fatto che tale valore sia un intero progressivo, un UUID o un altro tipo di riferimento.

La gestione di una richiesta di questo tipo comporta, lato server, due passaggi distinti: la risoluzione dell'identificatore in un oggetto concreto, tramite interrogazione del database o di un servizio dati, e il controllo che l'utente autenticato abbia diritto di accesso a quell'oggetto specifico. Il primo passaggio è quasi sempre implementato correttamente, poiché è indispensabile al funzionamento stesso dell'endpoint; il secondo, non avendo un effetto visibile quando manca, viene più facilmente trascurato o dato per scontato in fase di sviluppo.

Va osservato che l'endpoint, in questo scenario, non è di per sé vietato all'utente: la sua configurazione di ruoli e permessi lo autorizza a invocare quella funzione API. Questo distingue concettualmente BOLA da un'altra categoria della stessa famiglia, la Broken Function Level Authorization (BFLA), in cui è l'accesso alla funzione stessa a non essere autorizzato. In BOLA la funzione è legittima; il problema riguarda esclusivamente quale istanza dell'oggetto venga restituita o modificata.

Meccanismo dell'attacco Lo sfruttamento di una vulnerabilità BOLA non richiede, nella maggior parte dei casi, strumenti sofisticati né la compromissione di credenziali altrui. È sufficiente disporre di un account legittimo sul sistema bersaglio. Il percorso tipico dell'attacco segue una sequenza riconoscibile: l'utente accede al proprio account, richiede una propria risorsa attraverso l'interfaccia normale dell'applicazione e osserva, tramite gli strumenti di sviluppo del browser o un proxy di intercettazione, l'identificatore associato a quella risorsa nella richiesta sottostante.

A questo punto l'identificatore viene sostituito con un altro valore plausibile per lo stesso schema — un intero vicino, nel caso di sequenze numeriche, oppure un

³GraphQL: linguaggio di interrogazione per API in cui il client, anziché richiamare endpoint predefiniti, descrive in una sola richiesta l'esatto insieme di dati che desidera ricevere.

valore ottenuto per enumerazione più sistematica quando lo schema lo consente — e la richiesta viene reinviata con lo stesso token di autenticazione, valido e non modificato. Ad esempio:

```
GET /api/orders/483 HTTP/1.1
Host: api.example.com
Authorization: Bearer <token_utente_A>
```

dove 483 non è l'ordine appartenente al titolare del token, ma un identificatore vicino a quello osservato in precedenza durante il normale utilizzo dell'applicazione. La richiesta è, dal punto di vista del protocollo, del tutto regolare: cambia soltanto il valore del parametro. Che la risposta contenga davvero i dati di un altro utente dipende da un unico fattore, ripreso nel paragrafo seguente: se il server confronti o meno l'identità del chiamante con il proprietario effettivo dell'oggetto.

Esempio applicativo Si consideri il caso concreto della risorsa `/api/orders/{id}`, utilizzata per recuperare i dettagli di un ordine. Una richiesta legittima, effettuata dal proprietario dell'ordine 482, ha la forma:

```
GET /api/orders/482 HTTP/1.1
Host: api.example.com
Authorization: Bearer <token_utente_A>
```

e produce una risposta simile alla seguente:

```
HTTP/1.1 200 OK
Content-Type: application/json

{
  "order_id": 482,
  "customer_id": 1041,
  "items": ["..."],
  "shipping_address": "..."
```

La richiesta modificata introdotta nel paragrafo precedente, diretta all'ordine 483 e appartenente a `customer_id` 1078, viene gestita da un'implementazione vulnerabile allo stesso modo: l'identificatore viene estratto dal percorso, la tabella corrispondente viene interrogata, il risultato viene restituito. In nessun punto di questo flusso il valore di `customer_id` restituito viene confrontato con l'identità associata al token presente nell'header `Authorization`. La risposta, di conseguenza, è ancora un 200 OK, questa volta con i dati di un ordine che non appartiene al chiamante.

Il punto critico non è la struttura della richiesta, identica nei due casi, ma l'assenza di un confronto esplicito fra utente autenticato e risorsa richiesta. Un'implementazione corretta dovrebbe, prima di restituire l'oggetto, verificare che il `customer_id` dell'ordine coincida con l'identità del chiamante — o che quest'ultimo disponga di un diritto di accesso esplicitamente concesso su quella risorsa — e restituire, in caso contrario, un errore di autorizzazione: tipicamente 403 `Forbidden`, oppure 404 `Not Found` quando si preferisce non rivelare l'esistenza della risorsa a chi non ne ha diritto.

Cause e meccanismi tecnici tipici L'assenza di un controllo di autorizzazione a livello di oggetto è la causa immediata di ogni istanza di BOLA, ma raramente costituisce una scelta progettuale deliberata. Più spesso deriva da alcune condizioni che meritano di essere analizzate singolarmente.

La prima riguarda la sovrapposizione concettuale tra autenticazione e autorizzazione nella fase di implementazione. Un endpoint protetto da un middleware di autenticazione, che verifica la validità del token e ne estrae l'identità, può apparire sicuro a chi lo sviluppa, inducendo a trascurare la necessità di un controllo ulteriore specifico per l'oggetto acceduto. Il middleware risponde alla domanda “chi sta chiamando?”, non a “questo chiamante ha diritto su questo oggetto?”.

Una seconda causa è la fiducia implicita nei parametri forniti dal client. Alcune implementazioni derivano l'identità del proprietario di una risorsa dallo stesso identificatore fornito nella richiesta, oppure presuppongono che, poiché l'interfaccia utente dell'applicazione non espone mai identificatori altrui, un client “onesto” non li invierebbe mai. Questa assunzione ignora che un client HTTP può essere

costruito o modificato arbitrariamente da chi effettua la richiesta, indipendentemente dall'interfaccia grafica ufficiale.

Un terzo fattore è rappresentato da controlli di autorizzazione implementati in modo incompleto o incoerente tra i diversi endpoint di una stessa API. È relativamente comune riscontrare verifiche corrette sull'operazione di cancellazione di una risorsa e verifiche assenti sull'operazione di lettura o aggiornamento della medesima risorsa, poiché ciascun endpoint viene sviluppato e sottoposto a revisione in momenti diversi, con criteri non sempre uniformi. La crescita organica di un'API nel tempo, con l'aggiunta progressiva di nuovi endpoint da parte di sviluppatori diversi, tende ad amplificare questa disomogeneità.

Una causa più strutturale riguarda infine la gestione delle relazioni tra utenti e risorse nel modello dati. Quando tale relazione non è rappresentata in modo esplicito e interrogabile — ad esempio tramite una chiave esterna diretta tra la tabella degli utenti e quella delle risorse, o tramite un livello di autorizzazione centralizzato — ogni endpoint deve reimplementare autonomamente la logica di verifica, e questo aumenta la probabilità che qualche percorso venga trascurato.

Impatto L'impatto di una vulnerabilità BOLA dipende dalla natura della risorsa esposta, ma incide tipicamente su più proprietà di sicurezza contemporaneamente. Sul piano della riservatezza, l'accesso non autorizzato a un oggetto consente la lettura di dati che dovrebbero rimanere riservati al proprietario legittimo: informazioni personali, dettagli finanziari, contenuti sanitari, comunicazioni private, a seconda del dominio applicativo.

Quando l'endpoint vulnerabile consente anche operazioni di scrittura, l'impatto si estende all'integrità delle risorse: un attaccante può modificare dati appartenenti ad altri utenti, alterare lo stato di un ordine, cambiare le impostazioni di un account che non gli appartiene. Se l'operazione esposta è una cancellazione, la conseguenza diventa la perdita di disponibilità della risorsa per il legittimo proprietario.

Un aspetto che distingue BOLA da molte altre classi di vulnerabilità è la sua natura sistematicamente ripetibile: una volta individuato un endpoint privo del controllo appropriato, lo stesso schema di attacco può essere applicato a un numero

elevato di identificatori, trasformando quella che sembra una singola falla puntuale in un canale di accesso a porzioni estese della base dati.

Il rischio assume connotati particolari negli ambienti multi-tenant e cloud, dove più clienti condividono la stessa infrastruttura applicativa e la separazione tra i rispettivi dati dipende interamente dalla correttezza della logica di autorizzazione, non da una separazione fisica delle risorse. In un simile contesto, una singola vulnerabilità BOLA non compromette soltanto i dati di un utente, ma può erodere il confine logico tra tenant distinti, con implicazioni che vanno oltre il singolo incidente e toccano la fiducia complessiva riposta nell'infrastruttura condivisa.

Mitigazione La prevenzione di BOLA richiede che ogni accesso a una risorsa identificata da un parametro fornito dal client sia accompagnato da una verifica esplicita del diritto dell'utente su quello specifico oggetto, eseguita interamente lato server. Nessuna informazione proveniente dal client, incluso l'identificatore stesso della risorsa, può essere considerata prova di autorizzazione: l'identificatore indica quale oggetto viene richiesto, non chi ha diritto di accedervi.

Un principio applicabile in modo trasversale è quello del *least privilege*, secondo cui un soggetto dovrebbe disporre soltanto dei permessi strettamente necessari a svolgere le proprie funzioni, senza margini di accesso più ampi per comodità implementativa. Applicato al livello oggetto, questo si traduce nella necessità di associare esplicitamente ogni utente, o ruolo, alle risorse su cui ha effettivamente diritto di operare, evitando che l'accesso derivi da assunzioni implicite sulla struttura delle richieste.

Dal punto di vista architetturale, è preferibile centralizzare la logica di autorizzazione in un livello comune — un modulo, un middleware o un servizio dedicato — piuttosto che reimplementarla in modo indipendente in ciascun endpoint. Questa scelta non elimina la necessità di applicare il controllo in ogni punto di accesso, ma riduce la probabilità che errori o dimenticanze locali producano percorsi non protetti, e semplifica la verifica di coerenza tra le diverse funzioni esposte dall'API.

La coerenza dei controlli tra operazioni di lettura, scrittura e cancellazione sulla stessa risorsa merita infine attenzione specifica: la protezione di un'operazione non

implica automaticamente la protezione delle altre, ed è in questa asimmetria che si annida buona parte delle istanze reali di BOLA.

Le sottosezioni seguenti trattano in forma più sintetica le restanti categorie della OWASP API Security Top 10, a partire da API2:2023.

2.2.2 API2:2023 – Broken Authentication

Definizione e collocazione Broken Authentication occupa la seconda posizione della classificazione OWASP e si colloca concettualmente a monte di BOLA: mentre quest'ultima riguarda il controllo sull'accesso a uno specifico dato o risorsa, Broken Authentication compromette la capacità stessa del sistema di stabilire con certezza l'identità di chi effettua la richiesta. Un fallimento in questa fase rende vano qualsiasi controllo successivo, poiché un meccanismo di autorizzazione perde ogni efficacia se l'identità su cui si fonda può essere falsificata o aggirata [6]. A differenza di altre categorie più circoscritte, questa voce raggruppa un insieme eterogeneo di debolezze nei processi di verifica dell'identità: dalla gestione poco attenta dei token di sessione all'assenza di vincoli sul numero di tentativi di accesso consentiti. Un esempio comune riguarda i messaggi di risposta del server: se il sistema indica esplicitamente quale dei due campi, nome utente o password, sia errato, fornisce involontariamente all'attaccante una conferma sull'esistenza di quell'account. Parallelamente, l'assenza di limitazioni sulla frequenza delle richieste lascia l'interfaccia di login esposta ad attacchi automatizzati volti a indovinare le credenziali su larga scala.

Cause e meccanismi tecnici tipici Spesso queste problematiche nascono dal tentativo di implementare soluzioni di autenticazione proprietarie o non standard. Come evidenziato dalla documentazione OWASP, strumenti come le API key o protocolli di autorizzazione come OAuth vengono talvolta impiegati impropriamente per identificare gli utenti in scenari per cui non sono stati progettati, creando falle strutturali [6]. Un ulteriore fattore critico è la mancanza di controlli sul volume dei tentativi di accesso. Endpoint di autenticazione e flussi di recupero password privi di rate limiting risultano vulnerabili ad attacchi in cui elenchi di credenziali già compromesse altrove vengono provati in massa, sfruttando la tendenza diffusa a riutilizzare le medesime password su più servizi [7]. Infine, la gestione dei token di sessione (come i JSON Web Token, o JWT) introduce ulteriori insidie: errori nella convalida della firma crittografica, chiavi di cifratura deboli, mancato controllo dei tempi di scadenza o assenza di meccanismi di revoca immediata permettono a un attaccante di forgiare token validi o di continuare a utilizzare credenziali che dovrebbero essere

già decadute [8].

Impatto L'impatto di Broken Authentication è per sua natura sistemico: assumendo il controllo dell'identità di un utente, l'attaccante ne eredita tutti i privilegi operativi senza che il backend possa distinguere le azioni legittime da quelle malevole [6]. Ciò può tradursi non solo nell'accesso a dati riservati, ma anche nella manomissione dei parametri di sicurezza dell'account (come la modifica dell'indirizzo di recupero), trasformando un'intrusione temporanea nella perdita definitiva del profilo.

Mitigazione La contromisura principale consiste nell'affidarsi a standard industriali consolidati e a librerie verificate, evitando soluzioni "fai-da-te" per la gestione di token e password [9]. A livello applicativo e infrastrutturale è indispensabile introdurre politiche di rate limiting dedicate e restrittive sugli endpoint di login, adottare l'autenticazione a più fattori (MFA) dove possibile e imporre una nuova conferma delle credenziali prima di eseguire modifiche critiche sul profilo utente [10].

Rilevabilità Sul piano della rilevabilità l'analisi automatica si divide in due direttrici, a seconda della natura del difetto.

1. **Analisi dei token e della crittografia.** Le debolezze che riguardano i JWT e gli altri formati di sessione dipendono da regole deterministiche e ben definite, come la mancata verifica della firma o l'uso di algoritmi insicuri. Gli strumenti riescono quindi a individuarle con buona affidabilità, sia analizzando staticamente il codice sorgente per intercettare configurazioni errate delle librerie crittografiche, sia generando dinamicamente token manipolati per verificare se il backend li accetti [11, 12].
2. **Flussi di autenticazione e abusi comportamentali.** L'assenza di rate limiting e l'esposizione a tentativi di accesso massivi richiedono invece un monitoraggio a runtime: più che da un'ispezione sintattica del codice, queste carenze emergono dall'analisi dinamica del traffico o dai registri di sistema, dove si manifestano come picchi anomali di richieste fallite concentrate in brevi finestre temporali [7].

2.2.3 API3:2023 – Broken Object Property Level Authorization

Definizione e collocazione La *Broken Object Property Level Authorization* (BOPLA) identifica una classe di vulnerabilità nella quale i controlli di autorizzazione applicati alle API non considerano adeguatamente le singole proprietà degli oggetti gestiti dall'applicazione. Il problema non riguarda quindi soltanto la possibilità per un utente di accedere a un determinato oggetto, ma anche quali attributi di quell'oggetto possa effettivamente leggere o modificare [4].

La distinzione rispetto alla *Broken Object Level Authorization* (BOLA) è rilevante. Nel caso di BOLA, il controllo di autorizzazione fallisce a livello dell'oggetto: un utente riesce, per esempio, ad accedere a una risorsa appartenente a un altro utente manipolando l'identificativo della risorsa. Nel caso di BOPLA, invece, l'utente può avere legittimamente accesso all'oggetto, ma ottenere o modificare proprietà che non rientrano nei suoi privilegi. L'oggetto, quindi, non costituisce necessariamente la risorsa a cui viene negato l'accesso; è una sua specifica proprietà a essere esposta o manipolabile senza un'adeguata autorizzazione.

Il rischio è particolarmente rilevante nelle API REST, dove è frequente che un endpoint restituisca direttamente la rappresentazione serializzata di un oggetto. Una struttura interna composta da numerosi attributi può così diventare, in assenza di una selezione esplicita, il corpo stesso della risposta inviata al client.

Cause e meccanismi tecnici tipici Una delle cause più comuni è la sovrapposizione tra il modello utilizzato internamente dall'applicazione e quello esposto attraverso l'API. Un oggetto memorizzato nel database può contenere, ad esempio, proprietà quali `id`, `name`, `email`, `role` e `internalStatus`. Non tutte queste proprietà hanno però la stessa finalità né devono necessariamente essere accessibili allo stesso insieme di utenti.

Un'implementazione che serializza automaticamente l'intero oggetto può produrre una risposta contenente anche attributi che il client non dovrebbe conoscere. In questo caso il problema non deriva necessariamente da un accesso non autorizzato all'oggetto, ma dal fatto che l'API non applica una politica distinta alle proprietà che lo compongono. L'interfaccia grafica potrebbe nascondere determinati campi, ma

ciò non costituisce una misura di sicurezza: il contenuto della risposta HTTP rimane osservabile dal client e può essere analizzato direttamente.

Un secondo meccanismo riguarda le operazioni di scrittura. Framework e librerie possono consentire il *binding* automatico dei parametri ricevuti dal client verso gli attributi di un oggetto applicativo. Se il server accetta un payload come:

```
{
  "name": "Mario",
  "email": "mario@example.com"
}
```

un attaccante può provare ad aggiungere proprietà che non erano previste dal normale flusso applicativo:

```
{
  "name": "Mario",
  "email": "mario@example.com",
  "role": "admin"
}
```

Se il campo `role` viene accettato e aggiornato senza un controllo specifico, l'utente può modificare una proprietà interna che dovrebbe essere gestita esclusivamente dal sistema. Lo stesso principio può riguardare attributi quali `isAdmin`, `approved`, `price` o `ownerId`, a seconda del modello utilizzato dall'applicazione.

Il problema può quindi manifestarsi in due direzioni differenti: lettura non autorizzata di proprietà e modifica non autorizzata di proprietà. Nel primo caso l'attaccante osserva informazioni che non dovrebbe ricevere; nel secondo riesce a influenzare direttamente lo stato dell'applicazione attraverso attributi che non dovrebbero essere controllabili dal client.

Impatto L'impatto di una vulnerabilità BOPLA dipende principalmente dal tipo di proprietà coinvolta e dalle funzioni che la utilizzano. La semplice esposizione di un attributo può comportare una violazione della riservatezza, mentre la possibilità di modificarlo può compromettere l'integrità dei dati o il funzionamento dell'applicazione.

Nel caso di proprietà contenenti informazioni personali, finanziarie o relative all'organizzazione interna del sistema, l'esposizione può determinare una divulgazione di dati a soggetti non autorizzati. La possibilità di modificare determinate proprietà può invece portare alla perdita o alla corruzione dei dati. In situazioni più critiche, una proprietà utilizzata nei meccanismi di autorizzazione o nella gestione dello stato dell'account può essere sfruttata per ottenere un'escalation dei privilegi o contribuire alla compromissione dell'account. La gravità della vulnerabilità dipende quindi dal significato che quella proprietà assume all'interno dell'applicazione, più che dal meccanismo che ne consente l'accesso.

Mitigazione La mitigazione di BOPLA richiede che l'autorizzazione venga applicata non soltanto all'endpoint o all'oggetto, ma anche alle proprietà che possono essere restituite o modificate. La prima misura consiste quindi nel definire esplicitamente quali attributi ogni endpoint può esporre e quali possono essere accettati in input.

Per le risposte è preferibile evitare i meccanismi di serializzazione automatica, che restituiscono l'oggetto così com'è rappresentato internamente, e selezionare invece le proprietà necessarie alla singola operazione.

Per le operazioni di scrittura è necessario limitare esplicitamente gli attributi modificabili dal client. Una tecnica comune consiste nell'utilizzare una *whitelist* delle proprietà ammesse, evitando che il payload venga associato automaticamente a tutti gli attributi dell'oggetto.

Rilevabilità La rilevazione di BOPLA si articola in due verifiche distinte, corrispondenti alle due direzioni in cui una proprietà può attraversare il confine fra client e server.

La prima riguarda le proprietà esposte in risposta a una richiesta legittima, e si riduce al confronto fra gli attributi effettivamente restituiti e quelli previsti per il ruolo o il contesto dell'utente. È la verifica più diretta, perché l'evidenza è contenuta nella risposta stessa.

La seconda riguarda le proprietà accettate in ingresso, ed è più onerosa perché il server non le dichiara. Individuarle richiede di modificare il payload delle ri-

chieste, eventualmente ricorrendo a tecniche di *fuzzing*⁴ per scoprire attributi non documentati, e di verificare poi se il server ne recepisca effettivamente il valore.

Le due verifiche non sono sostituibili l'una con l'altra: un'API può accettare in input una proprietà sensibile che non restituisce mai, oppure restituire dati riservati senza consentirne la modifica. Un'analisi che si limitasse a una sola direzione lascerebbe scoperta metà del problema.

2.2.4 API4:2023 – Unrestricted Resource Consumption

Ogni richiesta servita da un'API consuma risorse: banda, CPU, memoria, storage e, con frequenza crescente, servizi a pagamento di terze parti. API4:2023 raccoglie i casi in cui almeno uno dei limiti che dovrebbero governare quel consumo è assente o dimensionato male, fra cui i timeout di esecuzione, la memoria allocabile e il numero di processi [13]. Non è il difetto di una funzione specifica, ma una proprietà trasversale di qualunque endpoint: i limiti di consumo sono la difesa in profondità di altre categorie, e la loro assenza si intreccia con gli endpoint di autenticazione di API2, con l'automazione dei flussi di business di API6, con la copertura incompleta della superficie esposta di API9 e con le chiamate verso servizi esterni di API10.

La radice comune è un'asimmetria: produrre una richiesta costa al client quasi nulla, servirla può costare al server una quantità che cresce in funzione di parametri che il client controlla, e finché quella funzione di costo non ha un limite superiore l'API resta esposta. I meccanismi sono tre. Il volume puro, dove più insidiosa dell'assenza del limite è la sua presenza in una sola dimensione, perché un tetto per indirizzo IP cade contro un attacco distribuito e uno per endpoint non intercetta il batching. La moltiplicazione di operazioni dentro una richiesta sola, illustrata dallo scenario GraphQL in cui centinaia di mutazioni aggregate in un unico batch saturano la memoria aggirando sia il limite di frequenza sia quello sulla dimensione delle immagini [13, 14]. Infine il costo algoritmico della singola richiesta [15], con il ReDoS come istanza più diffusa, documentata nei server JavaScript dove il thread singolo rende una richiesta sufficiente a congelare il processo [16]. Vi si aggiunge

⁴*Fuzzing*: tecnica di collaudo che sottopone al sistema grandi quantità di input generati automaticamente, spesso malformati o inattesi, per osservarne le reazioni anomale.

la causa che l'edizione 2023 porta in primo piano, l'assenza di soglie di spesa verso i fornitori esterni, dove la risorsa non è computazionale ma economica. Che limiti tanto semplici manchino dipende da due fatti: possono risiedere nel framework, nel proxy, nel gateway⁵ o presso il fornitore cloud, e quando ogni livello presume che se ne occupi un altro non se ne occupa nessuno; e la loro assenza non produce errori, sicché un endpoint privo di limiti supera ogni test funzionale.

L'esito immediato è il denial of service per esaurimento, che CWE-770⁶ enumera come conseguenza principale sulla disponibilità [17], ma il punto di rottura è la logica applicativa anziché la banda: una sola richiesta ReDoS⁷ o un solo batch GraphQL bastano, e nessuna difesa perimetrale basata sul volume li riconosce come anomali.

Nella mitigazione nessun livello basta da solo: i limiti fisici su memoria, CPU e processi sono dichiarati alla piattaforma, mentre all'ingresso dei dati va fissata una dimensione massima per ogni parametro e payload, vincolo che il contratto OpenAPI rende verificabile da uno strumento. Il rate limiting, segnalato al client con il codice 429 [18, 19], va tarato per endpoint e affiancato da un limite sulle esecuzioni della singola operazione per utente, che il conteggio delle chiamate non cattura [13], e si irrigidisce sugli endpoint di autenticazione [6, 9]; NIST lo colloca nel gateway anziché nel singolo servizio [20], scelta corretta purché ogni host esposto stia dietro quel gateway. Per la famiglia algoritmica servono timeout e motori a tempo lineare [16], per GraphQL l'analisi statica del costo di una query prima di eseguirla [14], per la dimensione economica soglie di spesa o allarmi di fatturazione.

Sulla rilevabilità le due fonti primarie sembrano contraddirsi: OWASP la giudica facile, perché richieste con parametri di dimensione crescente e l'osservazione di stato, tempo e lunghezza della risposta bastano a far emergere il problema [13], mentre CWE-770 giudica di utilità limitata l'analisi statica per le allocazioni prive di limite [17]. La contraddizione è apparente, perché OWASP descrive un tester in *black*

⁵*Gateway*: componente posto davanti ai servizi applicativi che riceve tutte le chiamate esterne e vi applica funzioni comuni quali autenticazione, instradamento e limiti di frequenza.

⁶CWE (*Common Weakness Enumeration*): il catalogo pubblico delle debolezze software mantenuto da MITRE; ogni voce, indicata dalla sigla CWE seguita da un numero, descrive una classe di difetto indipendentemente dal prodotto in cui compare.

⁷ReDoS (*Regular expression Denial of Service*): un input costruito ad arte che fa degenerare il tempo di valutazione di un'espressione regolare, saturando la CPU con una richiesta sola.

*box*⁸ e CWE uno strumento che legge il codice. Sul versante dinamico l'evidenza è consolidata, dalle tecniche basate sulla generazione di molte richieste in un intervallo breve al rilevamento per iniezione di carico [17, 21], e il test differenziale osserva il sistema nel suo insieme, vedendo il limite ovunque sia applicato ma solo sull'host che interroga. Sul versante statico la letteratura si concentra su sottoclassi ben delimitate, il ReDoS e il costo delle query GraphQL, mentre manca un corpo di lavori sottoposti a revisione paritaria sul rilevamento dell'assenza di rate limiting, timeout o limiti di paginazione nel codice di API REST. Se ne ricava la natura del segnale: in una BOLA l'assenza del controllo nel gestore coincide con la vulnerabilità, mentre qui l'analisi del codice può affermare solo che a un dato livello il limite non c'è, non che il difetto esista, perché potrebbe risiedere nel gateway. È un'assenza locale, che diventa segnalazione fondata solo quando un secondo segnale la conferma, e lo scenario di API9 mostra il caso simmetrico, in cui il limite presente a un livello non copre tutti gli host esposti [22]: la correlazione fra segnali eterogenei non è dunque un raffinamento dell'analisi, ma la condizione perché un giudizio sia possibile.

2.2.5 API5:2023 – Broken Function Level Authorization

Un'API espone operazioni, e non tutte sono destinate a tutti. Creare un invito con privilegi amministrativi, esportare l'elenco degli utenti, cancellare un account altrui sono funzioni riservate a un ruolo, e il controllo che verifica quel ruolo prima di eseguirle è l'autorizzazione a livello di funzione: API5:2023 raccoglie i casi in cui tale controllo manca o è aggirabile, e un utente ordinario riesce a invocare funzioni riservate [23]. La differenza rispetto ad API1:2023 è di livello, ed è l'unica che OWASP fissi in modo prescrittivo: nella BOLA l'accesso all'endpoint è previsto e la violazione avviene manipolando l'identificatore dell'oggetto, mentre se l'attaccante raggiunge una funzione cui non dovrebbe accedere il caso ricade sotto la BFLA [5]. Il riferimento tassonomico è CWE-285 *Improper Authorization*, che MITRE sconsiglia però di usare per mappare vulnerabilità reali, rimandando alle voci figlie CWE-862 quando il controllo è assente e CWE-863 quando esiste ma decide male [24].

⁸*Black box*: modalità di analisi che osserva il sistema solo dall'esterno, attraverso le risposte alle richieste, senza accesso al codice sorgente; vi si contrappone l'analisi *white box*, che legge il codice.

Il controllo funzionale manca perché non ha un luogo obbligato: OWASP osserva che l'autorizzazione su una funzione è gestita tramite configurazione oppure a livello di codice, e che scrivere controlli corretti è confuso, date le molte gerarchie di ruoli e gruppi delle applicazioni moderne [23]. Vi si somma il valore predefinito, perché una rotta aggiunta a un controller eredita il routing e l'autenticazione, non l'autorizzazione: nasce raggiungibile e tale resta. Sfruttarla non richiede manipolazioni, bastano chiamate legittime verso un endpoint cui non si dovrebbe accedere, e la struttura prevedibile delle API rende il tentativo facile, sostituendo GET con PUT oppure `users` con `admins`: la stessa regolarità che rende un'API documentabile la rende enumerabile. Lo scenario di riferimento è un'applicazione ad accesso su invito, in cui l'app mobile recupera un invito già emesso con una chiamata del tutto ordinaria:

```
GET /api/invites/{invite_guid} HTTP/1.1
Host: api.example.com
Authorization: Bearer <token_utente_normale>
```

La risposta espone, fra gli altri campi, il ruolo associato all'invito. L'attaccante ne cambia allora metodo ed endpoint, riutilizzando lo stesso token e senza manipolare alcun identificatore:

```
POST /api/invites/new HTTP/1.1
Host: api.example.com
Authorization: Bearer <token_utente_normale>
Content-Type: application/json
```

```
{"email": "attaccante@example.com", "role": "admin"}
```

L'operazione è riservata agli amministratori, ma soltanto attraverso una console priva di controllo di funzione: il server la esegue [23]. Il controllo, in un certo senso, esisteva, ma stava nella console, nel fatto che solo un amministratore vedesse quel pulsante. Tre osservazioni spostano infine il difetto fuori dal singolo gestore: il percorso non dice se un endpoint sia amministrativo, perché le funzioni riservate stanno

spesso sotto `api/users` anziché sotto `api/admins` [23]; i verbi HTTP lasciati abilitati per inerzia riaprono l'operazione da una via non presidiata [25]; le versioni obsolete continuano a servire il gestore precedente al controllo [22].

OWASP colloca l'impatto tecnico nella fascia grave, con esiti che vanno dalla divulgazione alla perdita o corruzione dei dati, fino all'interruzione del servizio [23]. Il danno non scala però con il numero di identificatori, bensì con la potenza della funzione raggiunta: una BOLA restituisce un record per richiesta, mentre una BFLA su un endpoint di esportazione ottiene l'intera base dati con una chiamata sola; e poiché le funzioni riservate sono in prevalenza funzioni di scrittura, il difetto che espone l'esportazione degli utenti espone, sull'endpoint accanto, la loro cancellazione.

La prescrizione centrale riguarda il valore predefinito: negare ogni accesso e richiedere concessioni esplicite ai ruoli per ogni funzione [23], con un vantaggio che sta in come si manifesta l'errore, perché se il default è il diniego una dimenticanza produce un 403 che il primo test funzionale rivela, mentre se il default è il permesso la stessa dimenticanza non produce alcun segnale. Segue la revisione degli endpoint alla luce della logica di business e della gerarchia dei gruppi, lavoro che non ammette scorciatoie rispetto alla conoscenza del dominio. La terza è l'unica con una forma sintattica: i controller amministrativi devono ereditare da un controller astratto che applichi il controllo in base a ruolo o gruppo, misura che non aggiunge verifiche ma le riposiziona, perché ometterle richiede l'atto deliberato di collocare un gestore fuori dalla gerarchia, discontinuità visibile in revisione e cercabile da uno strumento. Nessuna delle tre risolve però la copertura, perché il controllo protegge solo la superficie servita dal codice che lo contiene [22, 25].

OWASP giudica facile la rilevabilità, perché le API sono strutturate e il modo di raggiungere una funzione è prevedibile [23], ma il giudizio riguarda metà del problema: formulare la chiamata candidata è facile, stabilire se la risposta fosse dovuta non lo è, perché un 200 OK attesta che l'operazione è stata eseguita, non che fosse legittimo eseguirla. Il contributo della letteratura non è ispezionare il codice ma ricostruire la specifica mancante, ricavandola dalle pagine raggiungibili da ciascun ruolo [26], dalla coerenza con cui i controlli sono applicati dentro una partizione per ruolo [27] o dal contesto di autorizzazione [28]. Un'alternativa rinuncia a inferire e scrive regole di sicurezza per le API REST, verificandole con fuzzing stateful [29],

mentre un lavoro recente affronta lo stesso problema in modalità black box [30].

2.2.6 API6:2023 – Unrestricted Access to Sensitive Business Flows

Esistono attacchi che non violano alcuna regola. Qui ogni singola chiamata è legittima: l'utente ha diritto di compierla, i controlli di autorizzazione funzionano, il codice si comporta come progettato, e il danno nasce dal numero. OWASP definisce vulnerabile l'endpoint che espone un flusso di business sensibile senza limitarne adeguatamente l'accesso, portando come esempi l'acquisto di un prodotto, la creazione di un commento e la prenotazione di una fascia oraria [31]. Segue l'affermazione che definisce davvero la categoria: il rischio di un accesso eccessivo cambia da settore a settore, e la creazione di post tramite script può costituire spam per un social network ed essere incoraggiata da un altro [31]. Nessuna proprietà del codice, del contratto o della configurazione rende dunque un endpoint vulnerabile ad API6: lo stesso identico endpoint è sicuro in un contesto e vulnerabile in un altro, e a decidere è il modello di business che gli sta dietro.

Coerentemente, la causa che OWASP indica non è tecnica: manca una visione d'insieme dell'API che ne supporti i requisiti di business [31]. Il difetto nasce nel raccordo fra chi progetta l'endpoint e chi conosce il valore economico di ciò che l'endpoint muove, e dove quel raccordo manca nessuno è nella posizione di accorgersene. Lo sfruttamento richiede di comprendere il modello di business, individuare i flussi sensibili e automatizzarne l'accesso, e soltanto l'ultimo passo produce traffico osservabile: il primo scenario lo illustra con una console da gioco annunciata in quantità limitata, acquistata da un programma eseguito il giorno del lancio e distribuita su indirizzi IP e località diverse [31], dove la distribuzione geografica non è un dettaglio di colore ma la contromisura dell'attaccante contro il limite per indirizzo, e spiega perché il rate limiting nella sua forma più semplice non basti.

Sull'impatto il documento OWASP è internamente incoerente: la tabella dei punteggi classifica l'impatto tecnico come moderato, mentre la descrizione degli impatti afferma che un impatto tecnico in generale non è atteso e porta come esempi di danno l'impossibilità per gli utenti legittimi di acquistare un prodotto e l'inflazione nell'economia interna di un videogioco [31]. È la seconda formulazione a corrispondere agli

scenari descritti, che non producono né violazione di riservatezza né alterazione di dati né indisponibilità del servizio: ciò che cambia sta fuori dal sistema e si misura in scorte sottratte, ricavi persi o costi di acquisizione pagati per utenti inesistenti. Ne segue un effetto collaterale rilevante, perché un incidente che non produce indisponibilità né perdita di dati non attiva le procedure di risposta e viene percepito come problema commerciale anziché di sicurezza.

La mitigazione proposta ha una struttura a due livelli in cui l'ordine conta: prima si identificano, sul piano del business, i flussi che un uso eccessivo può danneggiare, e solo dopo si sceglie il meccanismo tecnico adeguato al rischio individuato [31]. Nessuna categoria precedente prescrive una fase preliminare simile, perché in nessuna altra la configurazione corretta dipende dall'azienda che la adotta. I meccanismi elencati, dal fingerprinting del dispositivo al captcha, dall'individuazione di comportamenti non umani come il passaggio dal carrello all'acquisto in meno di un secondo fino al blocco dei nodi di uscita Tor, sono presentati come modi per rallentare le minacce automatizzate, non per impedirle, e la logica che li accomuna è economica: nessuno produce una decisione binaria affidabile, tutti spostano il costo dell'attacco fino a superare il profitto atteso.

Il rilevamento automatico si scontra infine con l'assenza di un oracolo⁹: nessun artefatto analizzabile, che sia codice, contratto o configurazione, risponde alla domanda se un dato flusso in una data azienda vada protetto, e la mancanza di una mappatura CWE registra il fatto che non esiste una debolezza del software da nominare. Le tecniche note per i difetti di logica applicativa cercano deviazioni da un comportamento atteso [32, 33], mentre qui il flusso viene percorso correttamente e l'attacco sta nel ripeterlo. Restano le contromisure comportamentali a runtime indicate da OWASP [31], che riconoscono l'automazione durante l'attacco anziché la vulnerabilità prima che avvenga.

⁹*Oracolo*: in collaudo del software, il criterio che permette di stabilire se un dato comportamento osservato sia corretto o difettoso.

2.2.7 API7:2023 – Server Side Request Forgery

Il traffico non viaggia in una sola direzione: verso servizi interni, archivi remoti e risorse indicate da chi interroga il servizio partono richieste che l'applicazione stessa emette, ed è su queste che agisce l'attaccante in API7:2023. Il difetto compare quando un'API recupera una risorsa remota senza validare l'URL fornito dall'utente: l'attaccante può allora costringere l'applicazione a inviare una richiesta verso una destinazione inattesa, anche se protetta da un firewall o da una VPN [34]. Il server diventa un intermediario involontario, che agisce per conto di chi lo interroga ma con i propri privilegi di rete; non viola quindi un controllo di accesso applicativo, ne annulla uno di rete. MITRE classifica CWE-918 come debolezza *resultant*, che emerge cioè in presenza di altre [35]: l'SSRF è raramente il fine dell'attacco, quasi sempre il mezzo. A renderla attuale è il contesto in cui le API operano: è più comune, perché webhook¹⁰, recupero di file da URL e anteprime dei collegamenti spingono a raggiungere risorse esterne indicate dall'utente, ed è più pericolosa, perché fornitori cloud, Kubernetes e Docker espongono canali di gestione su HTTP a percorsi noti, mentre limitare il traffico in uscita resta difficile [34]. Rispetto ad API10:2023, con cui condivide il terreno del traffico in uscita e il divieto di seguire ciecamente le redirezioni, il criterio di distinzione è chi scelga la destinazione: l'attaccante in API7, un fornitore legittimo in API10 [36].

La causa immediata è l'assenza o l'inadeguatezza della validazione dell'URI fornito dal client, e sono i modelli di sviluppo moderni a incoraggiare l'accesso a URI di provenienza esterna: il difetto nasce quindi da una funzionalità voluta, perché nessuno introduce per sbaglio un endpoint che scarica un'immagine da un indirizzo scelto dall'utente. I due scenari del documento mostrano le due forme dell'attacco. Nella forma cieca un social network consente di indicare l'immagine del profilo tramite URL, e l'attaccante vi scrive un indirizzo interno:

```
POST /api/profile/upload_picture HTTP/1.1
Host: api.example.com
Content-Type: application/json
```

¹⁰Webhook: meccanismo per cui un servizio notifica un evento inviando una richiesta HTTP a un URL scelto e registrato dall'utente.

```
{"picture_url": "http://localhost:8080"}
```

Misurando il tempo di risposta deduce se la porta sia aperta, e scandisce così la rete interna [34]: l'informazione non arriva dal corpo della risposta, ma dalla sua latenza. Nella forma con ritorno un prodotto di sicurezza invia una richiesta di prova al webhook che l'utente sta configurando e gliene mostra la risposta, e l'attaccante vi indica l'indirizzo link-local del servizio di metadati dell'ambiente cloud — `http://169.254.169.254/latest/meta-data/` nella convenzione più diffusa — ottenendone le credenziali [34]: il webhook rende comune il difetto, l'indirizzo a percorso noto lo rende grave. Gli esiti vanno dall'enumerazione dei servizi interni per scansione di porte alla divulgazione di informazioni, dall'aggiramento di firewall all'uso del server come proxy [34]. L'elemento che distingue l'SSRF è il cambio di perimetro: l'attaccante non ottiene un dato, ottiene una posizione, quella del server, e il valore dell'attacco dipende perciò da cosa sia raggiungibile dall'interno. I servizi di metadati cloud convertono poi l'SSRF da ricognizione a compromissione, perché rispondono a un indirizzo noto senza autenticazione ed espongono le credenziali del ruolo associato alla macchina [35].

Sul piano della rete OWASP raccomanda di isolare il meccanismo di recupero delle risorse, che di norma mira a risorse remote e non interne: è la difesa più robusta perché non dipende dalla correttezza della validazione, dato che se il componente che effettua le richieste in uscita non ha rotta verso la rete interna un URL malevolo non produce nulla. Sul piano applicativo sono prescritte liste di elementi ammessi su tre dimensioni — origini remote attese, schemi di URL e porte, tipi di media accettati — ciascuna delle quali blocca una famiglia di aggiramenti diversa, cui seguono la disabilitazione delle redirezioni, un parser di URL collaudato e il divieto di restituire al client le risposte grezze [34]: quest'ultima non impedisce l'attacco, ma ne degrada l'esito dalla forma con ritorno a quella cieca. Va infine registrata un'ammissione che nessun'altra categoria contiene: il rischio SSRF non può sempre essere eliminato del tutto [34], perché funzionalità e vulnerabilità coincidono.

Per la prima volta nel capitolo le due fonti primarie concordano, e a favore dell'analisi statica: OWASP giudica facile la rilevabilità, con la sola eccezione della forma

cieca [34], e CWE-918 indica come metodo l’analisi statica automatica — la ricerca dei percorsi che collegano una sorgente di input a un *sink*¹¹ — dichiarandone alta l’efficacia [35]. Il motivo è che qui l’oracolo è interno al codice: per API4 e API5 lo strumento osserva l’assenza di un controllo senza poter sapere se fosse dovuto, mentre per API7 la domanda non è se qualcosa manchi, ma se due punti del programma siano connessi, proprietà verificabile senza conoscere le intenzioni del progettista. La letteratura conferma il quadro e ne segnala i limiti ricorrenti, dall’analisi di flusso validata manualmente [37] alla potatura dei percorsi non sfruttabili per contenere i falsi positivi [38], fino al fuzzing affiancato alla marcatura dinamica [39]. Se ne ricavano tre problemi che anche il modulo descritto nella parte sperimentale deve affrontare: l’inventario dei sink, la sovra-marcatura delle stringhe contaminate che non producono un URL controllabile, e la validazione intermedia, dove distinguere un controllo efficace da uno aggirabile richiede di ragionarne la semantica. Nella verifica dinamica, infine, la forma con ritorno si conferma puntando l’endpoint verso un indirizzo controllato, mentre quella cieca richiede un canale laterale, tipicamente una risoluzione DNS: nella pipeline adottata nel seguito la mancata conferma non declassa quindi la segnalazione statica.

2.2.8 API8:2023 – Security Misconfiguration

Un controllo che manca, un limite non impostato, un URL non validato: ciascuna delle categorie precedenti indica un punto preciso in cui il difetto risiede. API8:2023 non ha un punto: OWASP dichiara che la configurazione insicura può verificarsi a qualunque livello dello stack, dalla rete all’applicazione, e i suoi otto indicatori attraversano tutti quei livelli, dall’hardening¹² assente ai verbi HTTP lasciati abilitati, dal TLS¹³ mancante ai messaggi di errore che espongono tracce dello stack [25]. Hanno una sola cosa in comune: nessuno è un errore di logica applicativa, perché il codice

¹¹*Sorgente e sink*: nell’analisi del flusso dei dati, la *sorgente* è il punto in cui un valore controllato dall’esterno entra nel programma e il *sink* è l’operazione sensibile in cui quel valore viene infine usato.

¹²*Hardening*: insieme delle operazioni con cui si riduce la superficie di attacco di un sistema: disattivazione di funzioni e servizi non necessari, rimozione delle credenziali predefinite, applicazione delle impostazioni consigliate.

¹³TLS (*Transport Layer Security*): il protocollo che cifra e autentica il canale di comunicazione su cui viaggia il traffico HTTP.

fa quello che deve ed è l'ambiente in cui gira a produrre l'esposizione. Lo conferma la particolarità tassonomica: fra le sette voci CWE associate alla categoria compare CWE-16 *Configuration*, di cui MITRE dichiara proibita la mappatura, invitando ad analizzare piuttosto il comportamento che la configurazione ha determinato [40]. Se ne ricava il criterio: la configurazione non è una debolezza, è il meccanismo attraverso cui una debolezza viene introdotta, sicché una politica CORS permissiva non è insicura in astratto, lo diventa perché produce un difetto di controllo di accesso.

La causa strutturale è che la configurazione non risiede in un luogo unico né sotto una responsabilità unica: si distribuisce fra codice, file di orchestrazione, gateway, impostazioni cloud e valori predefiniti delle librerie, ciascuno con un ciclo di vita proprio, e un valore corretto al rilascio può cessare di esserlo dopo un aggiornamento che nessuno ha collegato alla sicurezza. Il primo scenario mostra il predefinito pericoloso, ed è riconoscibilmente il caso Log4Shell: un'utilità di logging supporta le ricerche JNDI abilitate per impostazione predefinita, e quando espande un'intestazione contenente un riferimento JNDI a un server LDAP controllato dall'attaccante ne scarica ed esegue l'oggetto malevolo [25], senza che nella catena vi sia alcun difetto del codice applicativo. Il secondo mostra il meccanismo opposto, l'impostazione assente: un social network espone i messaggi privati tramite `/dm/user_updates.json` e, non inviando l'intestazione `Cache-Control`, lascia che le conversazioni finiscano nella cache del browser, e qui non c'è nulla da disabilitare, c'è una direttiva che nessuno ha inviato. Resta poi la causa più diffusa, che l'elenco mette al secondo posto: patch mancanti e sistemi non aggiornati [25], difetto di manutenzione più che di configurazione.

Le configurazioni insicure, scrive OWASP, non solo espongono dati sensibili degli utenti, ma anche dettagli del sistema che possono condurre alla compromissione completa del server [25]: è la seconda parte a giustificare l'impatto severo, e i due scenari ne mostrano gli estremi, conversazioni private da un lato ed esecuzione di codice arbitrario dall'altro. Ciò che distingue API8 è però la natura abilitante del difetto, che raramente costituisce l'attacco e quasi sempre la condizione che ne rende praticabile un altro: una traccia di stack non compromette nulla di per sé, ma rivela versioni e percorsi interni e abbassa il costo della ricognizione, mentre una politica di uscita permissiva trasforma un SSRF cieco in uno con ritorno. Vi si aggiunge

l'ampiezza, perché un difetto di configurazione, essendo trasversale, non riguarda un endpoint ma tutti quelli serviti da quel componente.

Sul piano del processo OWASP chiede un hardening ripetibile, una revisione delle configurazioni sull'intero stack e un processo automatizzato che ne valuti l'efficacia in tutti gli ambienti, la cui implicazione comune conviene rendere esplicita: la configurazione va trattata come artefatto versionato e verificabile, non come stato accumulato di una macchina. Sul piano dei controlli le misure riguardano il canale cifrato verso ogni componente, i verbi HTTP da restringere, la politica CORS e le intestazioni di sicurezza, i tipi di contenuto accettati e gli schemi dei payload di risposta, comprese quelle di errore [25]; due meritano una nota, perché il canale cifrato è prescritto anche per le API interne, il che chiude l'assunto per cui il traffico interno sarebbe fidato, e lo schema dichiarato per le risposte di errore non è documentazione, ma un vincolo che a runtime impedisce la fuga di informazioni diagnostiche.

OWASP classifica la rilevabilità come facile con una motivazione che, per la prima volta nel capitolo, rinvia agli strumenti: ne esistono di automatici per individuare e sfruttare configurazioni errate [25], formulazione simmetrica perché gli stessi strumenti servono a chi difende e a chi attacca, dato che ciò che cercano è pubblicamente noto. La ragione per cui il rilevamento funziona si comprende per contrasto: per l'autorizzazione funzionale lo strumento non sapeva se un controllo fosse dovuto e per il consumo di risorse non sapeva se un limite fosse applicato altrove, mentre per la configurazione l'oracolo è esterno e pubblico — valori raccomandati, predefiniti noti come pericolosi, versioni vulnerabili — e verificare la conformità a un riferimento è un problema di confronto, non di inferenza. Restano tre limiti: la copertura, perché ciò che non è dichiarato in un file non è ispezionabile prima dell'esecuzione; la deriva, perché un'analisi statica attesta ciò che un file dichiara, non ciò che è attivo sul sistema; e il contesto, perché una politica CORS che ammette qualunque origine è un difetto per un'API a client autenticati e una scelta legittima per un'API pubblica di sola lettura, sicché la verifica sintattica stabilisce solo se un'impostazione si discosti da un riferimento la cui pertinenza dipende dall'endpoint.

2.2.9 API9:2023 – Improper Inventory Management

Correggere un difetto presuppone sapere che esiste il componente che lo ospita, e fin qui quel componente è sempre stato noto: un handler, un limite, una configurazione. API9:2023 riguarda invece ciò che l'organizzazione non sa di avere: il difetto non è nel codice ma nella visibilità, e OWASP lo definisce come scarsa catalogazione degli host e delle versioni di un'API, articolandolo in due punti ciechi [22]. Il primo è quello della documentazione, perché restano senza risposta esplicite le domande su quale ambiente ospiti un'API, chi debba avervi accesso di rete e quale versione sia in esecuzione, e non esistono né un piano di dismissione né un inventario aggiornato degli host. Il secondo è quello dei flussi di dati, e ricorre quando dati sensibili raggiungono terze parti senza motivazione di business, approvazione o visibilità su cosa venga condiviso [22]. Anche qui il riferimento tassonomico è una voce non mappabile, CWE-1059 [41]: come per API8, la categoria non nomina un comportamento del prodotto, e nel suo caso non è un'imprecisione della tassonomia ma una proprietà del difetto, perché l'assenza di un inventario non è qualcosa che il software faccia.

La causa è un'asimmetria fra due costi: esporre una nuova istanza richiede pochi minuti, catalogarla e prevederne la dismissione richiede un processo che nessuna pipeline esegue da sola, e ogni ambiente di collaudo è però un'API completa e raggiungibile, mentre l'inventario che dovrebbe elencarlo è un documento aggiornato a mano. Il primo scenario ne mostra la conseguenza: un social network protegge il recupero della password dalla forza bruta, ma il meccanismo non è nel codice dell'API, bensì in un componente interposto fra il client e l'host ufficiale, e un ricercatore individua `beta.api.socialnetwork.owasp.org`, che esegue la stessa API senza esserne coperto, e reimposta la password di qualunque utente indovinando un token di sei cifre [22]: il controllo esisteva ed era corretto, ma non copriva tutti gli host che servivano quel codice. Il secondo riguarda l'altro punto cieco, con un'applicazione malevola che ottiene il consenso di 270.000 utenti ma, attraverso di essi, accede alle informazioni private di 50.000.000 di contatti: qui non c'è alcun host dimenticato, manca la conoscenza di dove i dati finiscano.

OWASP classifica l'impatto tecnico come moderato, ma il dettaglio decisivo è

che versioni diverse della stessa API sono talvolta collegate alla medesima base di dati [22]: l'host dimenticato non è una copia isolata, è una seconda porta sugli stessi dati. Ne discende il tratto distintivo, perché API9 non introduce una vulnerabilità propria ma rende raggiungibili quelle altrui e ne impedisce la correzione, dato che una patch si applica a ciò che si sa di avere in esecuzione; il giudizio di impatto moderato è quindi fuorviante, perché la gravità effettiva è quella del difetto peggiore che l'host non inventariato continua a servire.

Le contromisure muovono tutte dallo stesso presupposto, che la visibilità vada prodotta e non presupposta. Le prime sono di inventario — gli host, con ambiente, accesso di rete e versione, e i servizi integrati, con ruolo e sensibilità dei dati — cui segue la documentazione dell'API, che deve coprire autenticazione, errori, redirezioni, limitazione di frequenza, politica CORS ed endpoint. La misura più incisiva è però la successiva: generare la documentazione in modo automatico, con standard aperti e dentro la pipeline di integrazione continua [22], differenza non di grado ma di natura, perché trasforma la documentazione da documento parallelo, che diverge appena qualcuno dimentica di aggiornarlo, in artefatto prodotto dalla compilazione. Restano le prescrizioni sulle versioni: applicare le misure di sicurezza esterne a tutte quelle esposte e non alla sola produzione, che è ciò che il primo scenario mostra disatteso, e non usare dati di produzione fuori dalla produzione.

OWASP giudica media la rilevabilità e non indica un metodo, limitandosi a osservare che una documentazione obsoleta rende più difficile individuare e correggere le vulnerabilità [22]: l'osservazione descrive un difetto che ostacola il proprio rilevamento, perché l'artefatto che servirebbe a trovarlo è quello che manca. Ciò che si può fare per via automatica è un confronto fra rappresentazioni: le rotte implementate nel codice ma non dichiarate nel contratto sono candidate a essere API ombra, mentre quelle dichiarate ma non implementate segnalano documentazione obsoleta, e nessuna delle due è una vulnerabilità, ma entrambe misurano la divergenza fra il modello e la realtà. Il confronto ha però un limite intrinseco, perché opera fra due rappresentazioni della stessa applicazione e non dice nulla sugli host su cui essa è distribuita: il primo scenario è precisamente il caso in cui codice e contratto coincidono e il difetto sta nella topologia di rete, il che richiede ricognizione infrastrutturale e non analisi del software. Per API9 non esiste dunque un rilevamento nel senso in cui

esiste per le altre categorie, cioè l'individuazione di un difetto dentro un artefatto: esiste la misura di una divergenza fra rappresentazioni indipendenti dello stesso sistema — contratto, codice, inventario, traffico — il cui valore non sta nel produrre un verdetto, ma nel restituire la visibilità che il difetto sottrae.

2.2.10 API10:2023 – Unsafe Consumption of APIs

Il perimetro di un servizio non coincide con il codice che lo compone: ogni dipendenza esterna è una superficie che non gli appartiene. API10:2023 rovescia perciò la prospettiva dell'attacco diretto, presupposta dalle nove categorie precedenti: l'attaccante non colpisce l'API bersaglio ma un servizio da cui essa dipende, e il danno arriva per un canale che l'applicazione considera fidato; le note di rilascio dichiarano di aver introdotto la categoria perché gli attaccanti hanno cominciato a compromettere i servizi integrati anziché le API del bersaglio [42]. La causa che OWASP individua è psicologica prima che tecnica: gli sviluppatori si fidano dei dati ricevuti da terze parti più di quanto si fidino dell'input dell'utente, e adottano standard più deboli quanto a trasporto, autenticazione e validazione [36]. Le cinque condizioni di vulnerabilità sono canale non cifrato, dati non validati, redirezioni seguite ciecamente, risorse non limitate e timeout assenti, e nel repository pubblico del progetto è stata sollevata, senza esito, l'obiezione che almeno tre di esse ricadano in altre categorie [43].

La causa strutturale è un confine di fiducia collocato nel punto sbagliato. Un servizio di terze parti occupa una posizione ambigua: è esterno all'applicazione, ma scelto deliberatamente, raggiungibile su un canale che essa ha configurato e spesso offerto da un'azienda reputata affidabile, e la sua risposta attraversa così un confine che nessuno ha presidiato, perché nessuno lo ha riconosciuto come tale. Il primo scenario mostra l'iniezione differita: un'API si affida a un servizio esterno per arricchire gli indirizzi commerciali degli utenti e ne salva i dati in una base di dati SQL, e gli attaccanti vi memorizzano un payload di SQL injection associato a un'attività creata da loro, poi inducono l'API a prelevarla [36]; ciò che lo distingue da un'iniezione ordinaria è che l'attaccante non invia mai il payload all'API bersaglio, e sanificare l'input degli utenti non serve, perché quell'input è un innocuo

nome di attività. Il secondo mostra la redirectione seguita ciecamente: un'API invia informazioni mediche a un fornitore su connessione cifrata, gli attaccanti lo compromettono ed esso risponde con un `308 Permanent Redirect`, sicché l'API ripete la richiesta identica, dati sensibili compresi, verso il loro server, e la fuga avviene per un comportamento predefinito del client HTTP che nessuno ha valutato come decisione di sicurezza. Il terzo scenario è il più istruttivo, perché la terza parte non deve nemmeno essere compromessa: un attaccante crea un repository Git denominato `' ;dropdb;--`, e l'applicazione che vi si integra ne usa il nome per costruire una query SQL ritenendolo sicuro [36], avendo confuso l'autenticità del canale con l'affidabilità del contenuto.

L'impatto varia a seconda di ciò che l'API bersaglio fa con i dati prelevati — esposizione di informazioni sensibili, molti tipi di iniezione o denial of service [36] — ed è classificato come severo. Ciò che caratterizza la categoria è che il difetto non ne determina l'entità, la quale è funzione di dove il dato non validato finisce: un'informazione prelevata da un fornitore e memorizzata può essere riletta mesi dopo da un componente che la tratta come dato interno, perché proviene dalla base di dati dell'organizzazione. Le contromisure prescritte sono quattro: valutare la postura di sicurezza dei fornitori al momento della scelta, imporre un canale di trasporto sicuro, validare e sanificare i dati ricevuti dalle API integrate, mantenere una lista delle posizioni verso cui esse possono indirigere [36]. La terza è il nucleo della categoria ed è formulata negli stessi termini riservati all'input dell'utente: la contromisura consiste nell'abolire la distinzione fra le due sorgenti. Due delle cinque condizioni — timeout e limiti sulle risposte — restano però prive di contromisura corrispondente; quando il contratto del fornitore è noto, la risposta può essere validata rispetto a uno schema, che non intercetta un payload conforme ma trasforma la fiducia implicita in un vincolo dichiarato.

OWASP classifica infine la rilevabilità come media senza indicare come procedere [36]. Sul piano statico il problema è di configurazione più che di tecnica: l'analisi della contaminazione richiede un insieme di sorgenti e uno di destinazioni, e per convenzione consolidata le sorgenti sono i punti in cui entra l'input dell'utente, mentre per API10 la sorgente è il valore restituito da una chiamata HTTP in uscita, che un motore configurato secondo quella convenzione non considera contaminato, sicché

un flusso dalla risposta di un fornitore a una query SQL non produce segnalazione benché sia identico a uno che partisse da un parametro di richiesta. Due dei cinque difetti si prestano invece a un rilevamento diretto, perché basta osservare la configurazione locale della chiamata: la redirectione seguita per impostazione predefinita e il timeout mancante, dove il segnale non è la presenza di un'opzione errata ma l'assenza di un'opzione che disattivi il predefinito insicuro. Sul piano dinamico la verifica richiede che il fornitore restituisca dati malevoli, il che è possibile solo se il servizio consente di immettervi contenuti oppure sostituendolo con un doppio in collaudo, che però verifica l'applicazione e non l'integrazione reale.

CAPITOLO 3

Metodo di ricerca

3.1 Approccio metodologico

3.1.1 Obiettivi e domande di ricerca

La rassegna del capitolo precedente restituisce un quadro disomogeneo: alcune categorie dell'OWASP API Security Top 10 dispongono di tecniche di rilevazione consolidate, altre sono trattate quasi solo in chiave dinamica, altre ancora non hanno una formulazione operativa che ne consenta il riconoscimento automatico. Il divario dipende da come si manifesta la violazione che definisce ciascuna categoria. Vi sono violazioni che lasciano una traccia strutturale negli artefatti di progetto — codice applicativo, definizioni infrastrutturali, specifica dell'interfaccia — e violazioni il cui riconoscimento presuppone di sapere quale uso dell'applicazione sia legittimo, informazione che nessun artefatto contiene.

Questa differenza corrisponde alla distinzione, già presente in letteratura, fra *vulnerabilità tecniche* e *vulnerabilità di logica di business* (*logic vulnerabilities*) [32, 33]. Le prime violano proprietà strutturali verificabili sugli artefatti di progetto; le seconde violano l'intento per cui il flusso applicativo è stato concepito. La stessa OWASP, nel motivare la categoria API6:2023, riconduce a questo carattere la resistenza dei difetti

di logica di business al rilevamento automatico: la correttezza è definita rispetto all'intento del dominio applicativo, non rispetto a una proprietà del codice [31].

Il presente lavoro applica sistematicamente tale distinzione alle dieci categorie dell'OWASP API Security Top 10 (2023) e ne verifica le conseguenze implementative con un'analisi multi-layer degli artefatti di progetto, integrata da evidenza dinamica dove la sola ispezione statica non basta. Su questa base si formula la domanda di ricerca principale.

Q RQ₁: *Quali categorie dell'OWASP API Security Top 10 (2023) sono riconducibili a vulnerabilità tecniche rilevabili mediante analisi statica multi-layer su codice applicativo, definizioni infrastrutturali e specifica dell'interfaccia, quali richiedono in aggiunta evidenza esterna agli artefatti, e di quale natura sia tale evidenza?*

La domanda si scompone in due interrogativi subordinati.

Q RQ_{1.1}: *Per le categorie riconducibili a vulnerabilità tecniche, quale combinazione di livelli di analisi è necessaria alla loro copertura, e quali condizioni di rischio emergono soltanto dalla congiunzione di segnali che, presi isolatamente, non costituiscono una violazione?*

Q RQ_{1.2}: *Per le categorie la cui proprietà violata non è interamente esprimibile sugli artefatti, quale componente resta riconoscibile staticamente e di quale natura è il residuo: accertabile sollecitando il sistema in esecuzione, colmabile estendendo il perimetro degli artefatti esaminati, esplicitabile come parametro dichiarato dell'analisi, oppure irriducibile alla conoscenza del dominio applicativo?*

3.1.2 Impostazione dello studio

Le domande non ammettono una risposta puramente documentale: stabilire quali categorie siano riconducibili a violazioni tecniche richiede un criterio applicato in modo sistematico, ma verificare che la riconduzione regga richiede di costruire i moduli di rilevazione corrispondenti e di osservarne il comportamento su codice reale. Lo studio assume perciò una forma costruttiva.

Ne discende il ruolo del sistema descritto nel seguito: è lo strumento con cui la classificazione viene messa alla prova, non l'oggetto della ricerca. Se una categoria è stata ricondotta a una violazione strutturale, deve esistere una regola che la riconosca; se è stata esclusa, l'assenza del modulo corrispondente è la conseguenza attesa dell'analisi. Si considerano evidenza a sostegno delle domande di ricerca soltanto la classificazione secondo il criterio dichiarato, l'esito verificato dell'esecuzione dei moduli su artefatti di prova e su un bersaglio realistico, e la documentazione motivata delle categorie non coperte; la presenza di regole nel codice non verificate su casi concreti non conta come copertura. Il capitolo formula il criterio di classificazione, ne deriva i requisiti per lo strumento e si chiude con il disegno della valutazione; l'applicazione del criterio alle dieci categorie è presentata insieme ai risultati.

3.1.3 Il criterio di classificazione

Il criterio rende operativa la distinzione della Sottosezione 3.1.1. Per ogni categoria si individua la proprietà di sicurezza la cui violazione la definisce, attenendosi alla formulazione ufficiale OWASP [4], alle debolezze CWE associate e alla caratterizzazione emersa dalla rassegna.

Una categoria è detta *tecnica* quando esiste un predicato sugli artefatti di progetto che sia al tempo stesso necessario e sufficiente a stabilire la violazione, una volta fissati i parametri dichiarati dell'analisi. È detta *ibrida* quando esiste un predicato necessario ma non sufficiente: l'analisi degli artefatti riconosce una condizione senza la quale la violazione non può sussistere, ma la cui presenza non basta a stabilirla. È detta infine *di logica di business* quando non esiste alcun predicato necessario esprimibile sugli artefatti, poiché la proprietà violata è definita per riferimento all'intento applicativo.

È necessario delimitare quali parametri sia lecito fornire all'analisi: si ammettono unicamente configurazioni generali valide per l'intero sistema (come l'uso di canali cifrati o limiti di traffico attesi). Si escludono invece regole puntuali sui permessi degli utenti rispetto alle singole risorse, per evitare di inserire a priori la logica di autorizzazione che l'analisi stessa dovrebbe verificare.

Il residuo delle categorie ibride non è omogeneo, e distinguerne la natura è l'oggetto di RQ_{1,2}. Un residuo è *dinamico* quando la proprietà mancante si manifesta

nella risposta del sistema a una sollecitazione mirata, tipicamente l’invio della stessa richiesta con identità distinte. È *di perimetro* quando è colmabile estendendo l’analisi a un tipo di artefatto non ancora esaminato: il limite riguarda l’applicazione dell’analisi statica a un solo livello, non l’approccio in sé. È *di dichiarazione* quando richiede che qualcuno espliciti la configurazione attesa o il contenuto di un accordo esterno. È *irriducibile* quando nessuna osservazione del sistema, statica o dinamica, può restituirlo.

Il criterio presuppone infine che il sistema da valutare sia noto. La condizione non vale per API9:2023, che descrive l’esistenza di sistemi che l’organizzazione ignora di possedere: la categoria resta fuori dalla tripartizione e viene trattata a parte.

3.1.4 Requisiti derivati per lo strumento di rilevazione

La classificazione non descrive uno strumento, ma vincola ciò che uno strumento adeguato deve saper fare. I requisiti che seguono sono formulati come capacità richieste, indipendentemente dalle tecnologie; la loro traduzione in un’architettura concreta è mostrata nella Sezione 3.2 per la Security Misconfiguration e nei capitoli seguenti per le altre categorie.

Il primo requisito è l’analisi su più livelli. Le proprietà individuate dal criterio non risiedono tutte nello stesso luogo: alcune si manifestano nel codice applicativo, altre nelle definizioni infrastrutturali, altre nella specifica dell’interfaccia, e nessun livello è da solo sufficiente. Uno strumento che esamini un solo tipo di artefatto non può quindi coprire il perimetro, e ciascun livello richiede una rappresentazione e una tecnica di ispezione adeguate al tipo di artefatto.

Il secondo requisito è la correlazione fra i livelli. Alcune condizioni non sono osservabili in alcun singolo artefatto, ma nella discrepanza fra due di essi: un percorso presente nel codice e assente dal contratto non è una violazione in nessuno dei due luoghi, eppure la divergenza segnala una superficie esposta e non governata; l’assenza di un limite nel codice cambia significato a seconda che le definizioni infrastrutturali ne prevedano uno altrove. Lo strumento deve quindi mettere in relazione segnali di livelli diversi e derivarne giudizi che nessun livello esprime da solo. Questo requisito risponde a RQ_{1.1}.

Il terzo requisito è la verifica dinamica: per le categorie che richiedono un riscontro a runtime, lo strumento deve poter inviare richieste reali all'API e analizzarne le risposte. L'analisi del codice rileva infatti solo un rischio potenziale, che rimarrebbe incerto senza una conferma attiva.

Il quarto requisito è la configurabilità dei parametri: i valori e le soglie utilizzati dall'analisi devono essere espliciti e modificabili dall'utente, per evitare che impostazioni predefinite e cablate nelle regole impongano scelte arbitrarie come se fossero vincoli universali.

Il quinto requisito è la tracciabilità: ogni segnalazione deve essere riconducibile alla categoria di cui costituisce evidenza e al livello di analisi che l'ha generata, perché una segnalazione priva di tale riconduzione può indicare un difetto reale ma non contribuisce a rispondere alle domande di ricerca.

3.1.5 Disegno della valutazione

La valutazione ha due scopi distinti: controllare che ogni modulo riconosca la condizione per cui è stato scritto, e osservare il comportamento dello strumento nel suo insieme su codice che non è stato scritto per essere analizzato.

Al primo scopo servono coppie di artefatti di prova, ciascuna formata da una versione che contiene la condizione da rilevare e da una che non la contiene: il modulo deve segnalare la prima e non la seconda. Le coppie coprono tutti e tre i livelli di analisi, così che nessun livello sia esercitato solo indirettamente attraverso il bersaglio realistico. Poiché artefatti e regole hanno lo stesso autore, questa prova stabilisce che l'implementazione fa ciò che si intendeva, non che funzioni su codice reale.

Al secondo scopo serve OWASP crAPI [44], applicazione volutamente vulnerabile mantenuta dalla OWASP Foundation e qui fissata a una revisione precisa per rendere i risultati riproducibili. Non essendo scritta dall'autore, riduce la circolarità della prima prova; resta però un bersaglio didattico, costruito perché le sue vulnerabilità siano riconoscibili, e i risultati non si possono estendere senza cautela ad applicazioni reali.

Per gli artefatti di prova il riferimento di verità è noto per costruzione; per crAPI è stato ricostruito a mano, individuando nel codice i punti in cui si manifestano le vulnerabilità documentate dal progetto, ed è riportato per esteso perché i risultati vadano letti rispetto a ciò che si è assunto come vero. Una segnalazione conta come corrispondente a una voce del riferimento quando riguarda la stessa categoria OWASP e lo stesso punto di esposizione dell'interfaccia, identificato da metodo e percorso normalizzato, qualunque sia il livello di analisi che l'ha prodotta; questa convenzione è stata fissata prima di osservare i risultati. Su questa base si registra, per ogni categoria, se i moduli segnalano gli artefatti che contengono la condizione e non quelli che ne sono privi, e si distinguono le categorie senza segnalazioni in cui ciò discende dalla classificazione da quelle in cui indica una copertura mancante.

3.2 Rilevamento delle Security Misconfiguration

I requisiti della Sottosezione 3.1.4 sono formulati come capacità; questa sezione mostra come sono stati resi operativi per la prima delle due voci in perimetro, la Security Misconfiguration della Sottosezione 2.1.1. La voce è quella che esercita di più il metodo: come si è visto, la configurazione non risiede in un luogo unico né sotto una responsabilità unica, e la stessa esposizione può essere dichiarata nel codice applicativo, nelle definizioni infrastrutturali o nella specifica dell'interfaccia, oppure manifestarsi soltanto nelle risposte del sistema in esecuzione. Richiede dunque tutti e tre i livelli di analisi previsti dal primo requisito e rende necessaria la correlazione richiesta dal secondo. La sezione descrive prima l'architettura entro cui i moduli operano, poi i tre livelli di analisi statica, quindi il modo in cui le segnalazioni vengono messe in relazione fra loro e con l'evidenza dinamica, e si chiude con la validazione condotta secondo il disegno della Sottosezione 3.1.5 e con i limiti che ne emergono.

3.2.1 Architettura del sistema

Il sistema è organizzato secondo i principi della *Clean Architecture*: un livello di dominio che definisce le entità e i contratti, un livello applicativo che orchestra

l'analisi, un livello infrastrutturale che incapsula gli strumenti esterni e un livello di presentazione che espone i risultati. La separazione risponde direttamente al quinto requisito, la tracciabilità: qualunque sia lo strumento che l'ha prodotta, ogni anomalia viene ricondotta a un'unica entità di dominio, il *Finding*, che registra la sorgente, la categoria, la severità su quattro livelli (CRITICAL, HIGH, MEDIUM, LOW, oltre a INFO per le note informative), un grado di confidenza fra 0 e 1, la posizione nel file, l'eventuale risorsa infrastrutturale o endpoint interessato, una chiave di correlazione e uno stato di convalida, che vale NOT_VALIDATED finché non interviene un riscontro dinamico e CONFIRMED dopo.

Due interfacce astratte fissano il contratto dei componenti di analisi. IScanner è implementata dagli *adapter* che avvolgono strumenti esterni — Checkov per le definizioni infrastrutturali [45], Spectral per la specifica dell'interfaccia [46], Sengrep per il codice [47], ZAP [48] e mitmproxy [49] per il traffico — e ha il compito di eseguire lo strumento, leggerne l'uscita e tradurla in Finding. IDetector è implementata dai moduli interni, che non invocano strumenti ma reagiscono a eventi. I componenti non si chiamano infatti a vicenda: comunicano attraverso un bus di eventi su cui l'orchestratore pubblica il completamento dell'analisi statica, la disponibilità del traffico catturato, ogni segnalazione rilevata e il completamento della pipeline. L'esecuzione procede in fasi: gli scanner statici vengono eseguiti sul repository bersaglio; l'evento di completamento sveglia i detector che lavorano sugli artefatti; se è disponibile traffico registrato, un secondo evento sveglia quelli che lavorano sulle risposte del sistema; le segnalazioni raccolte vengono separate in statiche e dinamiche in base alla sorgente e consegnate al motore di correlazione, che le unisce e assegna a ciascuna un punteggio di rischio. Aggiungere un controllo significa registrare un nuovo scanner o detector, senza modificare l'orchestratore.

Il modulo dedicato alla configurazione del codice applicativo, descritto nella sottosezione seguente, ha una forma più semplice: espone una funzione `analyze` che riceve il percorso del repository e restituisce un rapporto con le segnalazioni, un vettore che indica quali regole hanno prodotto almeno un risultato e un riepilogo per severità e categoria. Ogni segnalazione porta l'identificativo della regola, la debolezza CWE associata, la severità, il file e la riga, l'estratto di codice che costituisce l'evidenza, la protezione mancante espressa come indicazione correttiva e la confidenza. Il

modulo è invocato, accanto ai moduli dedicati alle altre categorie, dal runner unificato e dal cruscotto di presentazione.

3.2.2 Analisi del codice applicativo

Il modulo esamina il codice senza eseguirlo. Attraversa il repository saltando le directory di build, gli ambienti virtuali e le cache (`.git`, `.venv`, `node_modules`, `__pycache__`, `dist`, `build`) e trattiene i file con estensione `.py`, `.js` e `.ts`. I sorgenti Python vengono convertiti nell'albero sintattico astratto¹ messo a disposizione dal modulo `ast` della libreria standard, e ogni regola è realizzata come visitatore dell'albero che intercetta i nodi di suo interesse; per JavaScript e TypeScript, per i quali il modulo non dispone di un parser, l'unica regola attiva opera su espressioni regolari riga per riga. Su ciascun file vengono applicate quattro regole locali; una quinta è globale, perché la condizione che riconosce — l'assenza di una protezione — non è attribuibile a un file ma al repository nel suo insieme. Le segnalazioni con confidenza inferiore a 0,70 vengono scartate prima di comporre il rapporto: la soglia è il compromesso adottato fra copertura e rumore, e il suo effetto sulle singole regole è discusso più avanti. La Tabella 3.1 riassume le regole; i paragrafi che seguono ne descrivono il funzionamento.

La prima regola, SC-001, riconosce una politica CORS che ammette qualunque origine, condizione che annulla la *Same-Origin Policy* del browser e permette a un sito terzo di leggere le risposte dell'API [50]. Il riconoscimento è specifico per framework. In Flask la regola intercetta la chiamata `CORS(app, ...)` dell'estensione Flask-CORS e ne esamina l'argomento `origins`: il valore `"*"`, da solo o in una lista, produce una segnalazione con confidenza 0,95; l'assenza dell'argomento, che nell'estensione equivale ad ammettere qualunque origine [51], produce la stessa segnalazione con confidenza 0,75, perché la condizione è implicita e non dichiarata. In FastAPI e Starlette la regola intercetta `add_middleware(CORSMiddleware, ...)` e verifica `allow_origins`; se al carattere jolly si accompagna `allow_credentials=True`,

¹AST (*Abstract Syntax Tree*): rappresentazione ad albero della struttura sintattica di un programma, in cui ogni nodo corrisponde a un costrutto del linguaggio (una chiamata, un'assegnazione, una definizione di funzione); consente di riconoscere un costrutto per la sua forma e non per il testo che lo esprime.

Regola	CWE	Condizione riconosciuta	Severità	Confidenza
SC-001	942	Politica CORS che ammette qualunque origine (Flask-CORS, FastAPI/Starlette, Django, Express)	HIGH; CRITICAL se con credenziali	0,75 / 0,95
SC-002	94	Modalità di debug o ricaricamento automatico attivi nel codice o in un file di impostazioni	HIGH	0,95
SC-003	209	Gestore di errori che restituisce al client l'eccezione o la traccia dello stack	HIGH	0,90
SC-004	693	Nessun segnale di intestazioni di sicurezza nell'intero repository	MEDIUM	0,70
SC-005	798	Letterale stringa assegnato a una variabile dal nome sensibile	CRITICAL	0,40–0,90

Tabella 3.1: Regole di analisi del codice applicativo per la Security Misconfiguration. La confidenza è quella assegnata prima del filtro a 0,70.

la severità sale da HIGH a CRITICAL, perché la combinazione consente a un'origine arbitraria di inviare richieste con i cookie di sessione dell'utente. In Django la regola riconosce l'assegnazione `CORS_ALLOW_ALL_ORIGINS = True`; in Express riconosce `cors()` senza argomenti o con `origin: "*" .`

SC-002 riconosce la modalità di debug attiva in tre forme: la chiamata `run(..., debug=True)` sull'oggetto applicazione, che in Flask attiva la console interattiva di Werkzeug, capace di eseguire codice arbitrario sul server [52]; la chiamata `uvicorn.run` con `debug=True` o `reload=True`; l'assegnazione `DEBUG = True` nei file `settings.py`, `config.py` e `app.py`, dove le impostazioni di Django e delle applicazioni Flask sono per convenzione collocate. In tutti i casi la regola richiede il letterale booleano `True`: un valore letto dall'ambiente non viene segnalato, ed è esattamente la forma che la regola indica come correzione.

SC-003 riconosce i gestori di errore che restituiscono al client informazioni diagnostiche, la debolezza CWE-209 [53]. La regola individua le funzioni decorate con `@app.errorhandler`, prende il nome del parametro che riceve l'eccezione e ne segue l'uso nel corpo della funzione con un secondo visitatore. Tre forme di esposizione sono riconosciute, in ordine di gravità decrescente: l'invocazione di `traceback.format_exc()` o `print_exc()`, che riproduce l'intera traccia dello stack; `repr` applicato all'eccezione; `str` applicato all'eccezione o la sua interpolazione in una f-string. Quando più forme coesistono la segnalazione punta alla più grave,

con confidenza 0,90.

SC-004 è la regola globale. Cerca in tutti i file del repository, esclusi quelli di test, i segnali di una configurazione delle intestazioni di sicurezza: l'istanziamento di `Talisman` in `Flask` o di `helmet` in `Express`, la registrazione di un middleware tramite `add_middleware`, oppure la presenza letterale del nome di una delle quattro intestazioni difensive, cioè `Strict-Transport-Security`, `Content-Security-Policy`, `X-Frame-Options` e `X-Content-Type-Options`. Se nessun file contiene alcun segnale, la regola emette una sola segnalazione riferita al repository, con severità `MEDIUM` e confidenza 0,70; se anche un solo file ne contiene uno, non emette nulla. La forma aggregata evita che l'assenza di una protezione, che per sua natura non ha una riga, venga ripetuta per ogni file esaminato.

SC-005 riconosce le credenziali cablate nel codice, la debolezza CWE-798 [54]. Intercetta le assegnazioni di un letterale stringa a una variabile il cui nome contiene uno fra i termini sensibili (`secret`, `password`, `token`, `api_key`, `private_key`, `signing_key` e simili) e stima la confidenza dal valore assegnato. I valori più corti di otto caratteri ricevono 0,50, quelli presenti in una lista di segnaposto (`changeme`, `your_secret_here`, `placeholder`, `test`) 0,40, quelli che contengono spazi 0,45: tutti sotto la soglia, e quindi scartati. Per gli altri la confidenza dipende dall'entropia di Shannon [55] del valore v ,

$$H(v) = - \sum_{c \in v} p_c \log_2 p_c, \quad (3.2.1)$$

dove p_c è la frequenza del carattere c nella stringa: sopra 3,5 bit per carattere la confidenza è 0,90, sopra 2,5 è 0,70, altrimenti è 0,60 e la segnalazione viene scartata. L'entropia distingue una chiave generata casualmente, che usa molti caratteri con frequenze simili, da una parola o da un valore di prova; la severità è sempre `CRITICAL`, perché una credenziale nel repository è esposta a chiunque ne ottenga una copia.

Le regole SC-001 e SC-005 ignorano i file collocati in directory di test o di *fixture*, dove valori di prova e configurazioni permissive sono attesi; l'eccezione riguarda i soli artefatti di prova del sistema stesso, che devono restare analizzabili.

3.2.3 Analisi delle definizioni infrastrutturali

Per le definizioni infrastrutturali il sistema non implementa regole proprie ma si affida a Checkov [45], analizzatore statico per Infrastructure-as-Code che verifica i file Terraform [56] rispetto a un catalogo di policy sulle risorse cloud. L'adapter lo esegue come processo esterno con uscita in formato JSON, legge l'elenco dei controlli falliti e traduce ciascuno in un Finding. Due informazioni vengono derivate dal controllo. La prima è la categoria, ricavata dall'identificativo del controllo e dal tipo di risorsa: gestione delle identità, storage, rete, cifratura, logging, API gateway, oppure configurazione generica quando nessun indizio prevale. La seconda è la severità: **CRITICAL** per i controlli su identità e storage il cui identificativo indica esposizione pubblica, permessi con carattere jolly o privilegi amministrativi; **HIGH** per rete e cifratura e per ogni altro controllo che menzioni l'esposizione pubblica; **MEDIUM** negli altri casi. La confidenza è fissata a 1, perché il controllo di Checkov è deterministico rispetto al file. È a questo livello che ricadono i due scenari della Sottosezione 2.1.1 dichiarati in perimetro: i permessi di condivisione di una risorsa di storage sono contenuti nel manifest della risorsa, e lì vengono letti.

Il contributo specifico dell'adapter è la chiave di correlazione. Checkov identifica la risorsa con il nome Terraform, ad esempio `aws_api_gateway_method.order_post`; l'adapter ne estrae l'ultimo segmento, `order_post`, e lo registra come chiave. È il nome che lo sviluppatore ha assegnato alla risorsa, non il tipo generato dal provider, ed è quindi il nome con cui la stessa risorsa può essere ritrovata in altri artefatti e in altre segnalazioni.

3.2.4 Analisi della specifica dell'interfaccia

Il terzo livello è la specifica OpenAPI [57], esaminata con Spectral [46], un linter che valuta un documento rispetto a un insieme di regole dichiarative. L'adapter individua il contratto cercando fra i file YAML e JSON del repository quelli che dichiarano `openapi:` o `swagger:`, ed esegue Spectral con un insieme di regole che estende quello standard per la validità strutturale con tre controlli di sicurezza: il contratto deve dichiarare almeno uno schema di sicurezza in `components.securitySchemes`; ogni operazione di tipo `get`, `post`, `put`, `patch` o `delete` deve dichiarare il campo

`security`; ogni operazione deve avere una descrizione. I primi due sono violazioni di livello errore, tradotte in severità `HIGH`; il terzo è un avvertimento, tradotto in `MEDIUM`. L'assenza del campo `security` su un'operazione non prova che l'endpoint sia raggiungibile senza autenticazione — il controllo potrebbe risiedere in un gateway — ma stabilisce che il contratto non lo dichiara, ed è quanto un'analisi dell'artefatto può affermare.

L'adapter fa però una cosa in più, che anticipa la correlazione. Prima di eseguire Spectral confronta le rotte dichiarate nel contratto con gli endpoint che l'analisi del codice ha scoperto e costruisce una copia temporanea del contratto in cui gli endpoint presenti nel codice ma assenti dal contratto vengono aggiunti come operazioni minime. Il confronto avviene su percorsi normalizzati, così che `/users/<id>` nel codice e `/users/{userId}` nel contratto coincidano. Le violazioni sulle operazioni aggiunte vengono marcate come relative a un'API *scoperta* e non *documentata*: è la discrepanza fra codice e contratto di cui parla il secondo requisito, resa osservabile facendo passare attraverso le stesse regole anche ciò che il contratto tace.

3.2.5 Correlazione e punteggio di rischio

Il motore di correlazione riceve due insiemi di Finding, statici e dinamici, e li unisce in un unico inventario indicizzato per chiave. La chiave è quella esplicita quando l'adapter l'ha assegnata; altrimenti, per le segnalazioni riferite a un endpoint, è la coppia formata da metodo e percorso normalizzato; per le risorse infrastrutturali è l'identificativo della risorsa; in ultima istanza è la coppia file e riga. La normalizzazione dei percorsi sostituisce con il segnaposto `{id}` i segmenti numerici, gli UUID, gli hash esadecimali e i parametri di rotta nelle notazioni dei diversi framework (`<id>`, `{id}`, `:id`), ed elimina la stringa di query e il prefisso dell'API gateway locale: così la richiesta `/api/v1/users/12345` osservata nel traffico e la rotta `/api/v1/users/<int:id>` dichiarata nel codice ricevono la stessa chiave.

Le segnalazioni statiche vengono indicizzate per prime; se due condividono la chiave vengono fuse conservando la severità maggiore. Le segnalazioni dinamiche vengono poi confrontate con l'indice. Quando una trova una controparte statica, questa passa allo stato `CONFIRMED`, acquisisce l'evidenza runtime, sale di un livello

di severità (`HIGH` diventa `CRITICAL`, ogni altro livello diventa `HIGH`) e porta la confidenza a 1: la condizione prevista sull'artefatto è stata osservata nel comportamento del sistema. Quando non la trova, entra nell'inventario come segnalazione confermata a sé stante, con confidenza 0,9. È il terzo requisito reso operativo: l'analisi statica formula l'ipotesi, l'evidenza dinamica la conferma, e la distinzione fra le due resta leggibile nello stato della segnalazione.

Le segnalazioni delle cinque regole applicative non attraversano oggi questo motore: il modulo restituisce il proprio rapporto, che il runner unificato e il cruscotto presentano accanto a quelli degli altri moduli, mentre la correlazione opera sulle segnalazioni degli adapter infrastrutturale e contrattuale e su quelle dinamiche. Ne segue che una politica `CORS` aperta riconosciuta nel codice e un'intestazione `Access-Control-Allow-Origin: *` osservata nel traffico restano, allo stato attuale, due segnalazioni distinte; la loro unione è la naturale estensione del meccanismo appena descritto, non un meccanismo diverso.

Il punteggio di rischio segue una decisione documentata fra le scelte architetturali del sistema: un modello pesato proprio, preferito al CVSS perché quest'ultimo richiede la composizione di un vettore che nessuno degli strumenti integrati produce e non offre un modo naturale di rappresentare la conferma dinamica. Il punteggio è

$$R = \min(10, 0,6 S + 0,2 \cdot 10 C + 0,2 X), \quad (3.2.2)$$

dove S è il valore numerico della severità (9,0 per `CRITICAL`, 7,0 per `HIGH`, 4,5 per `MEDIUM`, 2,0 per `LOW`), $C \in [0,1]$ è la confidenza e X è un fattore di contesto che somma 4 punti se la risorsa è esposta a Internet, 4 se tratta dati sensibili e 2 se è dichiarata pubblica; quando nessuna informazione di contesto è disponibile, X vale 6 per le segnalazioni di autenticazione e autorizzazione e 3 per le altre. Pesi e costanti risiedono in un modulo di configurazione centrale e non nelle regole, come richiede il quarto requisito. Un esempio rende leggibile l'effetto della conferma: una segnalazione `HIGH` con confidenza 0,95 e contesto predefinito vale 6,7; la stessa segnalazione confermata a runtime, divenuta `CRITICAL` con confidenza 1, vale 8,0.

Ogni segnalazione conserva infine la protezione mancante come indicazione correttiva — per SC-002, leggere il flag dall'ambiente; per SC-005, trasferire il valore in una variabile d'ambiente — e un motore di remediation, che consulta una base di

conoscenza locale per i controlli Checkov, per le categorie OWASP e per le risorse cloud e, in sua assenza, interroga un modello linguistico eseguito localmente tramite Ollama, produce un'indicazione più estesa senza che alcun frammento di codice lasci la macchina dell'analista.

3.2.6 Validazione e limiti

La validazione segue il disegno della Sottosezione 3.1.5. Al primo scopo serve la coppia di artefatti di prova per il codice applicativo: un'applicazione Flask di trentaquattro righe che concentra le cinque condizioni — Flask-CORS con origine "*", `app.run(debug=True)`, un gestore di errori che restituisce `str(e)`, `traceback.format_exc()` e `repr(e)`, tre credenziali cablate, nessuna intestazione di sicurezza — e una controparte di trentadue righe che le inverte una per una: lista esplicita di origini, flag di debug letto dall'ambiente, gestore che registra l'eccezione nel log e restituisce un messaggio generico, credenziali lette dall'ambiente, `Talisman` istanziato. Il modulo segnala le cinque regole sulla prima e nessuna sulla seconda: cinque veri positivi, un vero negativo, nessun falso positivo né falso negativo, contro un criterio di accettazione che richiede una sensibilità di almeno l'ottanta per cento e nessun falso positivo. Come anticipato nel disegno, il risultato attesta che l'implementazione fa ciò che si intendeva: le due fixture sono state scritte dallo stesso autore delle regole, con la sintassi esatta che i visitatori cercano, e i valori non si estendono a codice non scritto per essere analizzato.

Al secondo scopo serve `crAPI` [44], nella revisione 1.1.5 fissata nel repository. L'applicazione è composta da servizi in linguaggi diversi: due in Python — il servizio *workshop*, un'applicazione Django, e il *chatbot*, un'applicazione Quart — uno in Go, uno in Java e un'interfaccia web in TypeScript. Il modulo produce quattro segnalazioni. Due sono di SC-002: `app.run(debug=True, port=5555)` nel *chatbot* e `DEBUG = True` nelle impostazioni del *workshop*. Una è di SC-005: l'assegnazione `API_GATEWAY_PASSWORD = "Pa$$4Vendor_1"` nello stesso file di impostazioni, che la regola distingue dalle altre credenziali del file, tutte lette dall'ambiente. Una è di SC-004, riferita all'intero repository: nessun sorgente configura intestazioni di sicurezza, e nemmeno i file di distribuzione le menzionano. Tutte e quattro corrispondono

a difetti presenti nel bersaglio.

Il confronto con il riferimento ricostruito a mano rivela però due falsi negativi, entrambi di SC-001, ed entrambi istruttivi. Il workshop abilita CORS verso qualunque origine con `CORS_ORIGIN_ALLOW_ALL = True`, nome storico dell'impostazione di `django-cors-headers` che la libreria accetta ancora accanto a quello attuale [58], mentre la regola riconosce soltanto `CORS_ALLOW_ALL_ORIGINS`. Il chatbot apre CORS con `cors(app, allow_origin="*")` dell'estensione `quart-cors`, la cui funzione ha nome minuscolo e argomento diverso da quelli di `Flask-CORS`. In entrambi i casi la condizione è dichiarata nell'artefatto ed è esattamente quella che la regola vuole riconoscere, ma la forma sintattica non è fra quelle previste: il limite non è del metodo, che qui è applicabile, ma del catalogo di forme che la regola conosce, e va registrato come copertura mancante nel senso della Sottosezione 3.1.5. A questi si aggiunge un limite di perimetro linguistico: i servizi in Go e in Java, che costituiscono la maggior parte del codice di `crAPI`, non vengono esaminati, perché il modulo dispone di un parser solo per Python. Il livello infrastrutturale, infine, non è esercitato su `crAPI`, che è distribuita con `Docker Compose`, `Helm` e manifest `Kubernetes` e non contiene definizioni `Terraform`; la sua verifica resta affidata alla coppia di artefatti di prova di quel livello.

L'uso su codice non scritto per essere analizzato fa emergere anche i limiti generali delle regole. SC-001 cerca un'apertura dichiarata: se il codice non configura CORS e la politica permissiva è impostata da un reverse proxy o da un gateway a monte, non c'è nulla da riconoscere. SC-002 richiede il letterale `True`, e non intercetta un flag impostato attraverso `app.config.update` o caricato da un file di ambiente tramite una libreria di impostazioni. SC-003 conosce il solo decoratore `errorhandler` di `Flask` e non i gestori di eccezione di `FastAPI` né i middleware `WSGI` o `ASGI` che formattano gli errori. SC-004 può sbagliare in entrambe le direzioni: produce un falso positivo quando le intestazioni sono iniettate da un componente perimetrale — una CDN, un gateway, un reverse proxy — che non compare nel repository, e un falso negativo quando conta come segnale la sola registrazione di un middleware, qualunque esso sia, dato che `add_middleware` è fra le firme cercate. SC-005 segnala un valore casuale assegnato a un nome sensibile anche quando è un identificativo di prova o un hash, e tace su una chiave strutturata con entropia inferiore a 2,5 o su una

credenziale assegnata a un attributo o a una chiave di dizionario anziché a un nome semplice. Vale infine un'osservazione sul quarto requisito: se i pesi del punteggio sono centralizzati, la soglia di confidenza a 0,70 e i valori di confidenza delle regole sono costanti del modulo, e non sono ancora esposti come parametri dell'analisi.

Questi risultati costituiscono l'evidenza che il disegno della valutazione ammette: l'esito verificato dell'esecuzione sugli artefatti di prova e su un bersaglio realistico, con il riferimento di verità dichiarato e le segnalazioni mancate registrate come tali. I due casi di CORS non riconosciuti su crAPI mostrano che la distanza fra la condizione definita e la sua rilevazione non sta nel criterio ma nella varietà delle forme con cui la stessa configurazione viene scritta, ed è su questa varietà che il catalogo delle regole va esteso.

CAPITOLO 4

Conclusioni

Le conclusioni qui...

Bibliografia

- [1] OWASP Foundation, “A05:2021 – security misconfiguration,” https://owasp.org/Top10/2021/A05_2021-Security_Misconfiguration/, 2021, oWASP Top 10:2021. [Online]. Available: https://owasp.org/Top10/2021/A05_2021-Security_Misconfiguration/ (Citato alle pagine 4 e 5)
- [2] SentinelOne, “Cloud misconfigurations: Risks, examples & prevention,” <https://www.sentinelone.com/cybersecurity-101/cloud-security/cloud-misconfigurations/>, 2025. [Online]. Available: <https://www.sentinelone.com/cybersecurity-101/cloud-security/cloud-misconfigurations/> (Citato a pagina 5)
- [3] Cloud Computing News, “10 real-life cloud security failures and what we can learn from them,” <https://www.cloudcomputing-news.net/news/10-real-life-cloud-security-failures-and-what-we-can-learn-from-them/>, 2025. [Online]. Available: <https://www.cloudcomputing-news.net/news/10-real-life-cloud-security-failures-and-what-we-can-learn-from-them/> (Citato a pagina 5)
- [4] OWASP Foundation, “OWASP API security top 10 – 2023,” <https://owasp.org/API-Security/editions/2023/en/0x11-t10/>, 2023. [Online]. Available: <https://owasp.org/API-Security/editions/2023/en/0x11-t10/> (Citato alle pagine 6, 17 e 39)

- [5] OWASP API Security Project, “API1:2023 broken object level authorization,” <https://owasp.org/API-Security/editions/2023/en/0xa1-broken-object-level-authorization/>, 2023, oWASP API Security Top 10 – 2023. Consultato il 25 agosto 2026. [Online]. Available: <https://owasp.org/API-Security/editions/2023/en/0xa1-broken-object-level-authorization/> (Citato alle pagine 8 e 22)
- [6] —, “API2:2023 Broken Authentication,” <https://owasp.org/API-Security/editions/2023/en/0xa2-broken-authentication/>, 2023, consultato il 25 agosto 2026. [Online]. Available: <https://owasp.org/API-Security/editions/2023/en/0xa2-broken-authentication/> (Citato alle pagine 15, 16 e 21)
- [7] OWASP Foundation, “Credential Stuffing,” https://owasp.org/www-community/attacks/Credential_stuffing, oWASP Community Pages. [Online]. Available: https://owasp.org/www-community/attacks/Credential_stuffing (Citato alle pagine 15 e 16)
- [8] Y. Sheffer, D. Hardt, and M. Jones, “JSON Web Token Best Current Practices,” RFC 8725, Internet Engineering Task Force (IETF), BCP 225, feb 2020. (Citato a pagina 16)
- [9] D. Temoshok, D. Proud-Madruga, Y.-Y. Choong, R. Galluzzo, S. Gupta, C. LaSalle, N. Lefkovitz, and A. Regenscheid, “Digital identity guidelines: Authentication and authenticator management,” National Institute of Standards and Technology, NIST Special Publication 800-63B-4, jul 2025. (Citato alle pagine 16 e 21)
- [10] OWASP Foundation, “Authentication Cheat Sheet,” https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html, oWASP Cheat Sheet Series. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html (Citato a pagina 16)
- [11] J. Yang, E. Wang, J. Chen, Q. Wang, Y. Zhang, H. Duan, W. Xie, and B. Wang, “Token Time Bomb: Evaluating JWT Implementations for Vulnerability Discove-

- ry,” in *Proceedings of the 32nd Network and Distributed System Security Symposium (NDSS)*. Internet Society, 2025. (Citato a pagina 16)
- [12] B. Xu, S. Jia, J. Lin, F. Zheng, Y. Ma, L. Liu, X. Gu, and L. Song, “JWTKey: Automatic Cryptographic Vulnerability Detection in JWT Applications,” in *Computer Security – ESORICS 2023*, ser. Lecture Notes in Computer Science, vol. 14346. Cham: Springer, 2023, pp. 263–282. (Citato a pagina 16)
- [13] OWASP API Security Project, “API4:2023 Unrestricted Resource Consumption,” <https://owasp.org/API-Security/editions/2023/en/0xa4-unrestricted-resource-consumption/>, 2023, consultato il 25 agosto 2026. [Online]. Available: <https://owasp.org/API-Security/editions/2023/en/0xa4-unrestricted-resource-consumption/> (Citato alle pagine 20 e 21)
- [14] A. Cha, E. Wittern, G. Baudart, J. C. Davis, L. Mandel, and J. A. Laredo, “A principled approach to GraphQL query cost analysis,” in *Proceedings of the 28th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE 2020)*. ACM, 2020, pp. 257–268. (Citato alle pagine 20 e 21)
- [15] S. A. Crosby and D. S. Wallach, “Denial of service via algorithmic complexity attacks,” in *Proceedings of the 12th USENIX Security Symposium*. Washington, DC: USENIX Association, 2003, pp. 29–44. (Citato a pagina 20)
- [16] C.-A. Staicu and M. Pradel, “Freezing the web: A study of ReDoS vulnerabilities in JavaScript-based web servers,” in *Proceedings of the 27th USENIX Security Symposium (USENIX Security 18)*. Baltimore, MD: USENIX Association, 2018, pp. 361–376. (Citato alle pagine 20 e 21)
- [17] MITRE Corporation, “CWE-770: Allocation of resources without limits or throttling,” <https://cwe.mitre.org/data/definitions/770.html>, 2026, cWE versione 4.20. Consultato il 25 agosto 2026. [Online]. Available: <https://cwe.mitre.org/data/definitions/770.html> (Citato alle pagine 21 e 22)
- [18] M. Nottingham and R. T. Fielding, “Additional HTTP status codes,” Internet Engineering Task Force, RFC 6585, apr 2012. (Citato a pagina 21)

- [19] R. Polli, A. Martinez Ruiz, and D. Miller, "RateLimit header fields for HTTP," IETF HTTPAPI Working Group, Internet-Draft draft-ietf-httpapi-ratelimit-headers-11, may 2026, work in progress. <https://datatracker.ietf.org/doc/draft-ietf-httpapi-ratelimit-headers/>. (Citato a pagina 21)
- [20] R. Chandramouli, "Security strategies for microservices-based application systems," National Institute of Standards and Technology, NIST Special Publication 800-204, aug 2019. (Citato a pagina 21)
- [21] J. Antunes, N. F. Neves, and P. Veríssimo, "Detection and prediction of resource-exhaustion vulnerabilities," in *Proceedings of the 19th IEEE International Symposium on Software Reliability Engineering (ISSRE 2008)*. IEEE, 2008. (Citato a pagina 22)
- [22] OWASP API Security Project, "API9:2023 Improper Inventory Management," <https://owasp.org/API-Security/editions/2023/en/0xa9-improper-inventory-management/>, 2023, consultato il 25 agosto 2026. [Online]. Available: <https://owasp.org/API-Security/editions/2023/en/0xa9-improper-inventory-management/> (Citato alle pagine 22, 24, 32 e 33)
- [23] —, "API5:2023 broken function level authorization," <https://owasp.org/API-Security/editions/2023/en/0xa5-broken-function-level-authorization/>, 2023, oWASP API Security Top 10 – 2023. Consultato il 25 agosto 2026. [Online]. Available: <https://owasp.org/API-Security/editions/2023/en/0xa5-broken-function-level-authorization/> (Citato alle pagine 22, 23 e 24)
- [24] MITRE Corporation, "CWE-285: Improper Authorization," <https://cwe.mitre.org/data/definitions/285.html>, common Weakness Enumeration. [Online]. Available: <https://cwe.mitre.org/data/definitions/285.html> (Citato a pagina 22)
- [25] OWASP API Security Project, "API8:2023 security misconfiguration," <https://owasp.org/API-Security/editions/2023/en/0xa8-security-misconfiguration/>, 2023, oWASP API Security Top 10 – 2023. Consultato il 25 agosto 2026. [Online]. Available: <https://owasp.org/>

- API-Security/editions/2023/en/0xa8-security-misconfiguration/ (Citato alle pagine 24, 29, 30 e 31)
- [26] F. Sun, L. Xu, and Z. Su, "Static detection of access control vulnerabilities in web applications," in *Proceedings of the 20th USENIX Security Symposium (USENIX Security 11)*. San Francisco, CA: USENIX Association, 2011. (Citato a pagina 24)
- [27] S. Son, K. S. McKinley, and V. Shmatikov, "RoleCast: Finding missing security checks when you do not know what checks are," in *Proceedings of the 2011 ACM International Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA 2011)*. ACM, 2011, pp. 1069–1084. (Citato a pagina 24)
- [28] M. Monshizadeh, P. Naldurg, and V. N. Venkatakrishnan, "MACE: Detecting privilege escalation vulnerabilities in web applications," in *Proceedings of the 2014 ACM SIGSAC Conference on Computer and Communications Security (CCS 2014)*. ACM, 2014, pp. 690–701. (Citato a pagina 24)
- [29] V. Atlidakis, P. Godefroid, and M. Polishchuk, "Checking security properties of cloud service REST APIs," in *Proceedings of the 13th IEEE International Conference on Software Testing, Validation and Verification (ICST 2020)*. IEEE, 2020, pp. 387–397. (Citato a pagina 24)
- [30] F. Liu, Y. Zhang, E. Li, W. Meng, Y. Shi, Q. Wang, C. Wang, Z. Lin, and M. Yang, "BACScan: Automatic black-box detection of broken-access-control vulnerabilities in web applications," in *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security (CCS 2025)*. ACM, 2025, pp. 1320–1333. (Citato a pagina 25)
- [31] OWASP API Security Project, "API6:2023 Unrestricted Access to Sensitive Business Flows," <https://owasp.org/API-Security/editions/2023/en/0xa6-unrestricted-access-to-sensitive-business-flows/>, 2023, consultato il 25 agosto 2026. [Online]. Available: <https://owasp.org/API-Security/editions/2023/en/0xa6-unrestricted-access-to-sensitive-business-flows/> (Citato alle pagine 25, 26 e 38)

- [32] V. Felmetsger, L. Cavedon, C. Kruegel, and G. Vigna, "Toward automated detection of logic vulnerabilities in web applications," in *Proceedings of the 19th USENIX Security Symposium (USENIX Security 10)*. Washington, DC: USENIX Association, 2010, pp. 143–160. (Citato alle pagine 26 e 37)
- [33] G. Pellegrino and D. Balzarotti, "Toward black-box detection of logic flaws in web applications," in *Proceedings of the 21st Network and Distributed System Security Symposium (NDSS 2014)*, San Diego, CA, 2014. (Citato alle pagine 26 e 37)
- [34] OWASP API Security Project, "API7:2023 server side request forgery," <https://owasp.org/API-Security/editions/2023/en/0xa7-server-side-request-forgery/>, 2023, oWASP API Security Top 10 – 2023. Consultato il 25 agosto 2026. [Online]. Available: <https://owasp.org/API-Security/editions/2023/en/0xa7-server-side-request-forgery/> (Citato alle pagine 27, 28 e 29)
- [35] MITRE Corporation, "CWE-918: Server-side request forgery (SSRF)," <https://cwe.mitre.org/data/definitions/918.html>, 2026, cWE versione 4.20. Consultato il 25 agosto 2026. [Online]. Available: <https://cwe.mitre.org/data/definitions/918.html> (Citato alle pagine 27, 28 e 29)
- [36] OWASP API Security Project, "API10:2023 Unsafe Consumption of APIs," <https://owasp.org/API-Security/editions/2023/en/0xaa-unsafe-consumption-of-apis/>, 2023, consultato il 25 agosto 2026. [Online]. Available: <https://owasp.org/API-Security/editions/2023/en/0xaa-unsafe-consumption-of-apis/> (Citato alle pagine 27, 34 e 35)
- [37] M. Wessels, S. Koch, G. Pellegrino, and M. Johns, "SSRF vs. developers: A study of SSRF-defenses in PHP applications," in *Proceedings of the 33rd USENIX Security Symposium (USENIX Security 24)*. Philadelphia, PA: USENIX Association, 2024, pp. 6777–6794. (Citato a pagina 29)
- [38] Y. Ji, T. Dai, Z. Zhou, Y. Tang, and J. He, "Artemis: Toward accurate detection of server-side request forgeries through LLM-assisted inter-procedural path-

- sensitive taint analysis,” *Proceedings of the ACM on Programming Languages*, 2025, preprint disponibile come arXiv:2502.21026. (Citato a pagina 29)
- [39] E. Wang *et al.*, “Where URLs become weapons: Automated discovery of SSRF vulnerabilities in web applications,” in *Proceedings of the 45th IEEE Symposium on Security and Privacy (S&P 2024)*. IEEE, 2024. (Citato a pagina 29)
- [40] MITRE Corporation, “CWE-16: Configuration,” <https://cwe.mitre.org/data/definitions/16.html>, 2026, categoria CWE, versione 4.20; mappatura vietata. Consultato il 25 agosto 2026. [Online]. Available: <https://cwe.mitre.org/data/definitions/16.html> (Citato a pagina 30)
- [41] —, “CWE-1059: Insufficient technical documentation,” <https://cwe.mitre.org/data/definitions/1059.html>, 2026, cWE versione 4.20; denominazione precedente *Incomplete Documentation* fino ad aprile 2022; mappatura vietata. Consultato il 25 agosto 2026. [Online]. Available: <https://cwe.mitre.org/data/definitions/1059.html> (Citato a pagina 32)
- [42] OWASP API Security Project, “Release notes – OWASP API security top 10 2023,” <https://owasp.org/API-Security/editions/2023/en/0x04-release-notes/>, 2023, consultato il 25 agosto 2026. [Online]. Available: <https://owasp.org/API-Security/editions/2023/en/0x04-release-notes/> (Citato a pagina 34)
- [43] —, “API10:2023 unsafe consumption of APIs – conflicting vulnerability indicators,” <https://github.com/OWASP/API-Security/issues/80>, 2023, issue n. 80 del repository GitHub OWASP/API-Security, marzo 2023. Consultato il 25 agosto 2026. [Online]. Available: <https://github.com/OWASP/API-Security/issues/80> (Citato a pagina 34)
- [44] OWASP Foundation, “crAPI: completely ridiculous API,” <https://github.com/OWASP/crAPI>, 2026. [Online]. Available: <https://github.com/OWASP/crAPI> (Citato alle pagine 41 e 50)
- [45] Prisma Cloud, “Checkov – what is checkov?” <https://www.checkov.io/>, 2026, analizzatore statico per Infrastructure-as-Code. [Online]. Available: <https://www.checkov.io/> (Citato alle pagine 43 e 47)

- [46] Stoplight, Inc., “Spectral – api style guide linter,” <https://docs.stoplight.io/docs/spectral>, 2026, linter per specifiche OpenAPI e AsyncAPI. [Online]. Available: <https://docs.stoplight.io/docs/spectral> (Citato alle pagine 43 e 47)
- [47] Semgrep, Inc., “Semgrep documentation,” <https://semgrep.dev/docs/>, 2026. [Online]. Available: <https://semgrep.dev/docs/> (Citato a pagina 43)
- [48] ZAP Dev Team, “ZAP – zed attack proxy documentation,” <https://www.zaproxy.org/docs/>, 2026. [Online]. Available: <https://www.zaproxy.org/docs/> (Citato a pagina 43)
- [49] mitmproxy Project, “mitmproxy documentation,” <https://docs.mitmproxy.org/>, 2026. [Online]. Available: <https://docs.mitmproxy.org/> (Citato a pagina 43)
- [50] MITRE Corporation, “CWE-942: Permissive cross-domain policy with untrusted domains,” <https://cwe.mitre.org/data/definitions/942.html>, common Weakness Enumeration. [Online]. Available: <https://cwe.mitre.org/data/definitions/942.html> (Citato a pagina 44)
- [51] C. Dolphin, “Flask-CORS documentation,” <https://flask-cors.readthedocs.io/>, 2026, estensione Flask per il Cross-Origin Resource Sharing. [Online]. Available: <https://flask-cors.readthedocs.io/> (Citato a pagina 44)
- [52] Pallets, “Werkzeug – debugging applications,” <https://werkzeug.palletsprojects.com/en/stable/debug/>, 2026, documentazione del debugger interattivo di Werkzeug. [Online]. Available: <https://werkzeug.palletsprojects.com/en/stable/debug/> (Citato a pagina 45)
- [53] MITRE Corporation, “CWE-209: Generation of error message containing sensitive information,” <https://cwe.mitre.org/data/definitions/209.html>, common Weakness Enumeration. [Online]. Available: <https://cwe.mitre.org/data/definitions/209.html> (Citato a pagina 45)
- [54] —, “CWE-798: Use of hard-coded credentials,” <https://cwe.mitre.org/data/definitions/798.html>, common Weakness Enumeration. [Online]. Available: <https://cwe.mitre.org/data/definitions/798.html> (Citato a pagina 46)

- [55] C. E. Shannon, "A mathematical theory of communication," *The Bell System Technical Journal*, vol. 27, no. 3, pp. 379–423, 1948. (Citato a pagina 46)
- [56] HashiCorp, "Terraform documentation," <https://developer.hashicorp.com/terraform/docs>, 2026. [Online]. Available: <https://developer.hashicorp.com/terraform/docs> (Citato a pagina 47)
- [57] OpenAPI Initiative, "Openapi specification v3.1.0," <https://spec.openapis.org/oas/v3.1.0.html>, 2021. [Online]. Available: <https://spec.openapis.org/oas/v3.1.0.html> (Citato a pagina 47)
- [58] A. Johnson, "django-cors-headers," <https://github.com/adamchainz/django-cors-headers>, 2026, middleware Django per il Cross-Origin Resource Sharing. [Online]. Available: <https://github.com/adamchainz/django-cors-headers> (Citato a pagina 51)

Ringraziamenti

Ringraziamenti qui...